

Generative AI in Sociology

Evidence, inference, and research design

Christopher Barrie

Fall 2026

Table of contents

1	Generative AI in Sociology	9
1.1	The question behind the course	9
1.2	Four parts of the semester	9
1.3	What this book contains	10
2	Course structure and schedule	11
2.1	A typical week	12
2.1.1	Weekly coding submission	13
3	How to use the book	14
3.1	Readings	14
3.2	Slides	14
3.3	Workbooks and code	14
3.4	Exercises	15
4	Recording and submitting weekly work	16
4.1	Deadline	16
4.2	Submit by email to Box	16
4.3	What the recording should show	16
4.4	Record on a Mac	17
4.5	Record on Windows	17
4.6	Record on a Chromebook	17
4.7	If your machine is different	18
5	Setting up Python, OpenRouter and Ollama	19
5.1	Six pieces of the setup	19
5.2	What runs where?	20
5.3	What OpenRouter is	20
5.3.1	What happens during an OpenRouter call	20
5.4	What Ollama is	21
5.4.1	What happens during an Ollama call	21
5.5	What is the same, and what differs?	21
5.6	What is actually sent?	22
5.7	Complete this before Week 1	23
5.8	Starting JupyterLab	23
5.9	Check the Ollama server before opening Python	24
5.10	Entering the OpenRouter key safely	24
5.11	The same research request through both routes	25
5.11.1	OpenRouter	25
5.11.2	Ollama	25
5.12	Use the model routine to learn Python objects	26

5.13	Choosing a route from Week 3	27
5.14	Cached outputs are contingencies	27
5.15	Safety and access rules	28
6	AI use: permitted with responsibility and disclosure	29
7	Assessment and marking	30
7.1	Weekly completion tasks	30
7.2	Midterm project proposal	30
7.3	Individual oral code walkthrough	31
7.4	Final project	31
7.5	Participation and reading preparation	31
I	Evidence, measurement and interpretation	32
8	Week 1 — What kind of research technology is an LLM?	33
8.1	This week’s sociological question	33
8.2	Connection to the preceding material	33
8.3	Learning outcomes	33
8.4	Essential readings and what each contributes	33
8.5	Additional readings grouped by purpose	34
8.6	Sociological and methodological background	34
8.7	Synthesis of the current LLM literature	34
8.8	Slides	34
8.9	Summary and preparation for next week	35
9	Week 1 Python — First calls and Python objects	36
9.1	Python walkthrough: make the first two model calls	36
9.1.1	Load the course settings and SDKs	36
9.1.2	Store the research material and construct one shared message list	37
9.1.3	Send the message list through OpenRouter and unpack its return	38
9.1.4	Send the same message list through Ollama and unpack its return	38
9.1.5	Use the request to learn strings, lists and dictionaries	39
9.1.6	Use the returns to learn objects, attributes and methods	40
9.1.7	Derive an integer and a Boolean from the returned strings	40
9.1.8	Preserve the two runs as separate research records	41
9.1.9	Methodological check	42
9.2	Workbook and environment links	42
9.3	Take-home exercise	42
10	Week 2 — When can an LLM label count as a measure?	43
10.1	This week’s sociological question	43
10.2	Connection to the preceding material	43
10.3	Learning outcomes	43
10.4	Essential readings and what each contributes	43
10.5	Additional readings grouped by purpose	44
10.6	Sociological and methodological background	44
10.7	Synthesis of the current LLM literature	44

10.8 Slides	44
10.9 Summary and preparation for next week	45
11 Week 2 Python — LLM annotation	46
11.1 Python walkthrough: use an llm as an annotator	46
11.1.1 Load the course settings and SDKs	46
11.1.2 Store one comment, the codebook and a human reference label	47
11.1.3 Construct the exact message list used by both routes	48
11.1.4 Make and unpack the OpenRouter call	48
11.1.5 Make and unpack the Ollama call	49
11.1.6 Compare route agreement and human-reference agreement	49
11.1.7 Methodological check	50
11.2 Workbook and environment links	50
11.3 Take-home exercise	50
12 Week 3 — What can an LLM contribute to qualitative interpretation?	51
12.1 This week's sociological question	51
12.2 Connection to the preceding material	51
12.3 Learning outcomes	51
12.4 Essential readings and what each contributes	51
12.5 Additional readings grouped by purpose	52
12.6 Sociological and methodological background	52
12.7 Synthesis of the current LLM literature	52
12.8 Slides	52
12.9 Summary and preparation for next week	53
13 Week 3 Python — Qualitative suggestions in JSON	54
13.1 Python walkthrough: ask for a qualitative suggestion in json	54
13.1.1 Load the course settings and SDKs	54
13.1.2 Choose one route and store two identified excerpts	55
13.1.3 Convert the excerpts into prompt text and construct messages	56
13.1.4 Make the selected route's call without hiding either branch	56
13.1.5 Parse the JSON string and return to its cited source	57
13.1.6 Methodological check	57
13.2 Workbook and environment links	58
13.3 Take-home exercise	58
14 Week 4 — What part of social experience becomes evidence?	59
14.1 This week's sociological question	59
14.2 Connection to the preceding material	59
14.3 Learning outcomes	59
14.4 Essential readings and what each contributes	59
14.5 Additional readings grouped by purpose	60
14.6 Sociological and methodological background	60
14.7 Synthesis of the current LLM literature	60
14.8 Slides	60
14.9 Summary and preparation for next week	61

15 Week 4 Python — Multimodal evidence	62
15.1 Python walkthrough: give a multimodal model visual evidence	62
15.1.1 Load the course settings and SDKs	62
15.1.2 Load the supplied image transport utility and identify two frames	63
15.1.3 State the evidentiary boundary and the required JSON fields	64
15.1.4 Send the images through the selected route	64
15.1.5 Parse the three fields and compare them with the images	65
15.1.6 Methodological check	66
15.2 Workbook and environment links	66
15.3 Take-home exercise	66
 II Experiments and research encounters	 67
16 Week 5 — Can generated media isolate a causal construct?	68
16.1 This week’s sociological question	68
16.2 Connection to the preceding material	68
16.3 Learning outcomes	68
16.4 Essential readings and what each contributes	68
16.5 Additional readings grouped by purpose	69
16.6 Sociological and methodological background	69
16.7 Synthesis of the current LLM literature	69
16.8 Slides	69
16.9 Summary and preparation for next week	69
 17 Week 5 Python — Generating experimental materials	 70
17.1 Python walkthrough: generate experimental materials	70
17.1.1 Load the course settings and SDKs	70
17.1.2 Choose a route and store the fixed facts and changing frame	71
17.1.3 Construct the prompt with an f-string	72
17.1.4 Make one timed call and preserve route-specific metadata	72
17.1.5 Save the design settings beside the candidate	73
17.1.6 Methodological check	73
17.2 Workbook and environment links	74
17.3 Take-home exercise	74
 18 Week 6 — What changes when a model starts asking the questions?	 75
18.1 This week’s sociological question	75
18.2 Connection to the preceding material	75
18.3 Learning outcomes	75
18.4 Essential readings and what each contributes	75
18.5 Additional readings grouped by purpose	76
18.6 Sociological and methodological background	76
18.7 Synthesis of the current LLM literature	76
18.8 Slides	77
18.9 Summary and preparation for next week	77

19 Week 6 Python — Adaptive interview turns	78
19.1 Python walkthrough: build one adaptive interview turn	78
19.1.1 Load the course settings and SDKs	78
19.1.2 Choose a route and construct the opening interview history	79
19.1.3 Make the first call through the selected route	80
19.1.4 Append the realized probe and a second participant answer	80
19.1.5 Make the second call with the expanded history	81
19.1.6 Methodological check	81
19.2 Workbook and environment links	82
19.3 Take-home exercise	82
 III Synthetic populations and social simulation	 83
20 Week 7 — Whose attitudes do synthetic respondents represent?	84
20.1 This week’s sociological question	84
20.2 Connection to the preceding material	84
20.3 Learning outcomes	84
20.4 Essential readings and what each contributes	84
20.5 Additional readings grouped by purpose	85
20.6 Sociological and methodological background	85
20.7 Synthesis of the current LLM literature	85
20.8 Slides	85
20.9 Summary and preparation for next week	86
 21 Week 7 Python — Structured synthetic responses	 87
21.1 Python walkthrough: request a structured synthetic response	87
21.1.1 Load the course settings and SDKs	87
21.1.2 Choose a route and store one deidentified profile and survey item	88
21.1.3 Define the exact output structure	89
21.1.4 Make the selected structured-output call	89
21.1.5 Parse, check structure and only then compare with the held-out answer . . .	90
21.1.6 Methodological check	91
21.2 Workbook and environment links	91
21.3 Take-home exercise	91
 22 Week 8 — Can LLMs recover opinions we did not observe?	 92
22.1 This week’s sociological question	92
22.2 Connection to the preceding material	92
22.3 Learning outcomes	92
22.4 Essential readings and what each contributes	92
22.5 Additional readings grouped by purpose	93
22.6 Sociological and methodological background	93
22.7 Synthesis of the current LLM literature	93
22.8 Slides	93
22.9 Summary and preparation for next week	94

23 Week 8 Python — Repeat predictions and compare routes	95
23.1 Python walkthrough: repeat one prediction across five cases	95
23.1.1 Load the settings and start with five records	95
23.1.2 Create two empty result lists	96
23.1.3 Follow one trip through the loop	96
23.1.4 Make both calls and keep the raw text	97
23.1.5 Parse and append one record to each list	97
23.1.6 Count exact matches, then compare the aggregate means	98
23.1.7 Methodological check	99
23.2 Workbook and environment links	99
23.3 Take-home exercise	99
24 Week 9 — How can micro-level interactions generate macro-level order?	100
24.1 Question	100
24.2 Why this week belongs here	100
24.3 Provisional core readings	100
24.4 Planned chapter and class arc	100
24.5 Development status	100
25 Week 9 Python — Putting a model call inside a function	101
25.1 Planned code trace	101
25.2 Planned completion recording	101
25.3 Development status	101
26 Week 10 — When can agent diversity and debate produce collective intelligence?	102
26.1 Question	102
26.2 Why this week belongs here	102
26.3 Provisional core readings	102
26.4 Planned chapter and class arc	102
26.5 Development status	102
27 Week 10 Python — Running a small interaction repeatedly	103
27.1 Planned code trace	103
27.2 Planned completion recording	103
27.3 Development status	103
IV Research agents, replication and model audits	104
28 Week 11 — Who—or what—produces research data in an agentic workflow?	105
28.1 Question	105
28.2 Why this week belongs here	105
28.3 Provisional core readings	105
28.4 Planned chapter and class arc	105
28.5 Development status	105
29 Week 11 Python — A transparent research-agent tool call	106
29.1 Planned code trace	106
29.2 Planned completion recording	106

29.3 Development status	106
30 Week 12 — What can we reproduce, and what remains opaque?	107
30.1 Question	107
30.2 Why this week belongs here	107
30.3 Provisional core readings	107
30.4 Planned chapter and class arc	107
30.5 Development status	107
31 Week 12 Python — Rerunning and auditing an agent trajectory	108
31.1 Planned code trace	108
31.2 Planned completion recording	108
31.3 Development status	108
32 Week 13 — What culture, ideology and hierarchy are encoded in models?	109
32.1 Question	109
32.2 Why this week belongs here	109
32.3 Provisional core readings	109
32.4 Planned chapter and class arc	109
32.5 Development status	109
33 Week 13 Python — A small model audit	110
33.1 Planned code trace	110
33.2 Planned completion recording	110
33.3 Development status	110
34 Week 14 — Project presentations	111
34.1 Presentation format	111
34.2 What to bring	111
35 References	112

1 Generative AI in Sociology

Evidence, inference, and research design

1.1 The question behind the course

What inference is the researcher trying to make, and what evidence would validate the use of an LLM for that inference?

Large language models can classify text, suggest interpretations, generate treatments, conduct interviews, imitate survey respondents, take part in simulations, search for evidence and write code. None of those abilities tells us, on its own, whether a sociological claim is warranted. That is the problem this course takes seriously.

The course is about 60 per cent method and 40 per cent substantive sociology of AI. We use LLMs as research tools, but also study the social assumptions, institutions and inequalities that become visible through them. The aim is not to turn sociologists into software engineers. It is to make the technical steps understandable enough that you can decide what the output means, what it does not mean, and what should be checked next.

No prior programming course is required. We introduce Python gradually through actual research tasks. Each example names its input, shows the operation, displays the output and then asks what evidence would validate the resulting claim. You will be expected to explain the code in plain language. Remembering package syntax is much less important than understanding what a value represents and how it changes.

1.2 Four parts of the semester

1. **Evidence, measurement and interpretation (Weeks 1–4).** We begin with research records, text annotation, qualitative analysis and multimodal observation.
2. **Experiments and research encounters (Weeks 5–6).** We examine generated treatments and interviews in which the next question depends on the preceding answer.
3. **Synthetic populations and social simulation (Weeks 7–10).** We ask what model-generated respondents and interacting agents can represent.
4. **Research agents, replication and model audits (Weeks 11–14).** We examine long-horizon research systems, reproducibility and LLMs as objects of sociological study.

The final project returns to the opening question. You will propose an LLM-related sociological study and show that its claims, data, model operations and validation plan fit together.

1.3 What this book contains

Each week is divided into two pages. The first gives you the sociological question, the readings and why they matter, the methodological background and the lecture slides. A separate Python page then gives you the worked input-to-output trace, complete workbook and take-home completion task. Week 14 is reserved for presentations and therefore has no Python page. The HTML edition is the canonical version: it is searchable and contains the live slide decks and downloads. The PDF is provided for reading and annotation away from the site.

2 Course structure and schedule

The seminar meets on Wednesdays from 9:30 a.m. to 12:15 p.m. The sequence is cumulative, but it is not a ladder of model capability. Each week introduces a different kind of sociological claim and asks what an LLM changes in the production of evidence.

Week	Date	Question
1	2 September	What kind of research technology is an LLM?
2	9 September	When can an LLM label count as a measure?
3	16 September	What can an LLM contribute to qualitative interpretation?
4	23 September	What part of social experience becomes evidence?
5	30 September	Can generated media isolate a causal construct?
6	7 October	What changes when a model starts asking the questions?
—	14 October	No Wednesday meeting; midterm project-proposal period
7	21 October	Whose attitudes do synthetic respondents represent?
8	28 October	Can LLMs recover missing or unobserved social worlds?
9	4 November	How can micro-level interactions generate macro-level order?
10	11 November	When can agent diversity and debate produce collective intelligence?
11	18 November	Who—or what—produces research data in an agentic workflow?

Week	Date	Question
12	25 November	What can we reproduce, and what remains opaque?
13	2 December	What culture, ideology and hierarchy are encoded in models?
14	9 December	Project presentations; no new reading, lecture or weekly task

2.1 A typical week

Before class, read the essential material with the chapter question in mind. The additional readings are routes into a debate, not an undifferentiated list.

The normal 165-minute meeting has three clearly different parts:

Time	What we do
9:30–10:00	The instructor introduces the sociological problem, relevant methodological background and the question connecting the readings.
10:00–10:50	Class discussion of the readings. We compare arguments, evidence and disagreements rather than hearing a second lecture summarizing each text. Come ready to identify one claim, one piece of evidence and one question.
10:50–11:00	Break.
11:00–12:05	Guided walkthrough of the weekly coding exercise. We build the LLM routine in short blocks, predict each result, run it, inspect the model return and explain the input and output of every operation.
12:05–12:15	Questions, recording/submission reminder and preparation for the following week.

The exact timing can move slightly when discussion is especially productive, but the middle of the class is reserved for collective discussion and the final part is reserved for working through the code together.

The coding strand teaches Python through actual LLM routines. Week 1 introduces values, strings and the anatomy of a call. From Week 2, students send a prompt to a model through OpenRouter or a supported local runtime. Later weeks add message lists, structured outputs, parsing, generation parameters, loops and, only after those are familiar, small functions with explicit parameters.

Cached responses remain available when a service is unavailable, but they are the contingency rather than the main route.

2.1.1 Weekly coding submission

The weekly completion task is submitted as a short **screen recording with audio**. Record your screen while you run the exercise and explain aloud:

1. the input to each operation and its Python type;
2. what the operation does;
3. the output it produces and its Python type;
4. what changed when you modified the named input or parameter;
5. what the returned model output does—and does not—show for the week’s research question.

Email the recording to Box by **5:00 p.m. Eastern on the Tuesday before the next class**. The [recording and submission guide](#) gives the upload address and instructions for macOS, Windows and ChromeOS. A polished video is not expected; a clear, continuous walkthrough is enough.

The book uses *AI and Research Methods* as a recurring map. It is not a substitute for the empirical papers. The chapter tells you which section to read and what question it helps answer. [Download the chapter PDF](#) or consult the [APSA Preprints record](#).

3 How to use the book

3.1 Readings

Start with the short description beneath each essential reading. It tells you why the reading is there and what to look for. Read the assigned section rather than assuming that every page is equally important. Additional readings are grouped by purpose—for example, historical background, a competing result or a technical extension.

The site links to DOI records, publishers or stable author copies. It does not redistribute copyrighted PDFs. If NYU access is required, use the library link resolver in a separate tab.

3.2 Slides

The full Reveal.js deck appears inside each HTML chapter and can be opened full screen. The deck is a teaching aid, while the chapter supplies the argument and transitions that should remain understandable away from the room. In the PDF edition, a link replaces the embedded deck.

3.3 Workbooks and code

The course assumes no prior Python knowledge. Each week has a separate **Python work** page immediately after its readings-and-slides page in the book navigation. Every required coding exercise is printed there in full. The code is divided into short blocks, and each block includes its exact input, a line-by-line explanation, the expected output and the research meaning of that output. This keeps the main weekly page focused on the readings and lecture while giving the code enough space to be explained properly.

The downloadable .py task file repeats the Python-page exercise exactly and provides a convenient place to run it. It does not contain an extra or more difficult assignment. The notebook is optional and provides further examples. Neither download substitutes for the explanation on the Python page. For every required block, be able to answer five questions:

1. What value enters, and what does it represent?
2. What is its Python type?
3. What operation is performed?
4. What value comes out?
5. What sociological claim could that output support, and what would still need checking?

Weeks 1–2 send the same synthetic or public teaching material through OpenRouter and Ollama. Students choose either route in Weeks 3–7 and compare both again in Week 8. A clearly labelled recorded return is supplied as a contingency if the hosted service, network or local runtime is unavailable; using that contingency does not reduce the completion mark.

Each developed week also has an **Open in Google Colab** link. Colab opens the notebook in a browser and supplies a temporary Python environment. Run the first setup cell once: it downloads the public `cjbarrie/GenAI_Soc2026` repository and moves to the correct weekly folder so that the notebook can find its accompanying data. To keep your changes, use Colab’s **File → Save a copy in Drive**; the temporary runtime itself is not permanent. Colab uses the OpenRouter route because it cannot normally connect to Ollama running on your own computer.

3.4 Exercises

Weekly tasks receive completion marks. Each task begins with complete, runnable code and asks you to change one clearly identified input or parameter. The Python page explains the possible outputs and what to inspect; a live model response may not be identical across students or runs.

Submit a short screen recording with audio. Your recording should show the code running and explain the input, operation and output of each block in ordinary language. Show the original run and the named change, then explain the methodological meaning and one limit of the output. It is due by **5:00 p.m. Eastern on the Tuesday before the next class**. The [recording and submission guide](#) explains how to record on macOS, Windows and ChromeOS and gives the Box upload email address.

You do not need to edit the recording or produce a polished presentation. The assessment is whether you understand the routine. If AI materially assists your preparation, say so briefly at the end of the recording using the disclosure described in the policy chapter.

In the developed early weeks, you will not be asked to define a Python function, interpret a test harness or make code more efficient. Those are useful programming skills, but they are not the purpose of these completion tasks.

4 Recording and submitting weekly work

4.1 Deadline

Upload each weekly coding recording by **5:00 p.m. Eastern on the Tuesday before the next class**. For example, the Week 1 recording from Wednesday 2 September is due on Tuesday 8 September at 5:00 p.m.

If the class schedule changes, a dated instruction on the weekly page or in an NYU announcement takes priority. Missing elements may be supplied within one week, as described in the [assessment policy](#).

4.2 Submit by email to Box

Email your recording to Coding_.38jln5sii39h6mf@u.box.com

Attach the recording to the email. Name the file `lastname_weekNN.mp4`, replacing `lastname` and `NN` with your surname and the two-digit week number. Files ending in `.mov` or `.webm` are also accepted. You do not need to write a message in the email body.

Wait until the attachment has finished uploading before sending. Keep the message in your Sent folder as confirmation of submission. Do not put the shared OpenRouter key in the filename, recording, notebook, subject line or email body.

4.3 What the recording should show

Make one short, continuous screen recording. Editing and camera video are not required. Your screen and voice should make it possible to follow:

1. the supplied code running on the original input;
2. the one input or parameter the task asks you to change;
3. the input and Python type used by each operation;
4. what each operation does and what it returns;
5. whether the model is reached through OpenRouter or Ollama;
6. what the output means for the week's research question;
7. one important limit, connected to an essential reading.

Enter the OpenRouter key **before** recording, or use Ollama, so the key never appears on screen. Before uploading, play the first and last few seconds of the file and check that your voice is audible.

4.4 Record on a Mac

1. Press **Shift–Command–5**.
2. Choose **Record Entire Screen** or **Record Selected Portion**. Recording only the notebook or terminal area helps keep other information private.
3. Select **Options**, then choose the microphone you want to use.
4. Select **Record** and talk through the exercise while it runs.
5. Use the stop button in the menu bar when you finish. Open the saved file and check the sound before uploading it.

These controls are part of macOS; no additional software is needed. See [Apple’s screen-recording instructions](#).

4.5 Record on Windows

The recommended route is **Clipchamp**, which is included with Windows 11:

1. Open Clipchamp and choose **Record & create**, then **Screen**.
2. Allow microphone access when asked and select the correct microphone.
3. Choose the notebook window, browser tab or screen you want to share.
4. Start recording and talk through the exercise.
5. Stop the recording, preview it with the sound on and export or save the video.

You do not need to use the camera or edit the result. If Clipchamp is unavailable, Windows 11’s Snipping Tool can record a selected part of the screen, but check that your version also captures your microphone before completing the exercise. See [Microsoft’s Clipchamp screen-recording instructions](#) and its [overview of Windows recording tools](#).

4.6 Record on a Chromebook

1. Press **Shift–Ctrl–Show windows**. If the keyboard has no Show windows key, press **Ctrl–Shift–F5**.
2. Select **Screen record**, then choose the full screen, a window or a selected area.
3. Open **Settings** in the recording controls and turn on **Microphone**.
4. Start recording and talk through the exercise.
5. Select **Stop recording** at the bottom right. The **.webm** file is normally saved in Downloads.
6. Open the file and check the sound before uploading it.

See [Google’s Chromebook screen-recording instructions](#).

4.7 If your machine is different

Use its built-in screen recorder or a private Zoom meeting if your NYU account permits local recording. The requirements remain the same: the code must be visible and your explanation must be audible. Test a ten-second recording before doing the exercise. Ask before the deadline if the available tools do not produce a usable file.

Your camera is not required. If spoken narration presents an accessibility problem, contact the instructor to arrange an equivalent submission format.

5 Setting up Python, OpenRouter and Ollama

The course uses real model calls from Week 1. Everyone configures both supported routes during the first two weeks. After that, you normally choose either OpenRouter or Ollama; Week 8 returns to a direct comparison.

This chapter separates several things that are easy to conflate: Python, a notebook, an SDK, an API, a server and a model. They work together, but they are not interchangeable names for the same thing.

5.1 Six pieces of the setup

Piece	What it is	What it does in this course
Python	a programming language and runtime	stores research material, constructs a request and handles the return
JupyterLab or VS Code	an interface for running Python	lets us execute the routine in short, inspectable steps
SDK or client library	an installed Python package	converts ordinary Python objects into an API request and converts the reply into a Python response object
API	a documented way for software to send requests to a service	specifies the endpoint, authentication, request fields and response fields
server	the program that receives the API request	OpenRouter operates remote servers; Ollama operates a server on the local computer
model	the trained system that generates the return	selected by a hosted model slug or a downloaded Ollama tag

Installing `openrouter` or `ollama` does not install Python, Jupyter or—in the OpenRouter case—the model. The packages are relatively small pieces of client software. Their job is to connect our Python code to a service.

5.2 What runs where?

Route	What your Python contacts	Where inference runs	What you need
OpenRouter	OpenRouter's hosted API	a remote provider selected through OpenRouter	internet access and the privately supplied course key
Ollama	a server at <code>localhost</code>	your computer	the Ollama application and downloaded model
Cached contingency	a saved response file	nowhere during the exercise	only when a live route is genuinely unavailable

Local JupyterLab or VS Code is the normal environment because it can reach both routes. Colab remains an OpenRouter fallback; it cannot normally run Ollama on your computer.

5.3 What OpenRouter is

OpenRouter is a hosted routing service. We send one request to OpenRouter's API and name a model using a string such as `"openai/gpt-5.2"`. OpenRouter then selects an available provider endpoint for that model, forwards the request and returns the result. The model name and the provider that actually serves a request are therefore related but distinct pieces of the research record.

The course uses OpenRouter because one SDK and one key let us work with hosted models without learning a different provider library for every model. That convenience does not remove the need to record routing information, cost, data policy or the resolved model/provider when the service supplies it.

5.3.1 What happens during an OpenRouter call

1. Python stores the prompt and messages in memory on the student's computer.
2. The `openrouter` SDK turns those Python objects into an HTTPS request.
3. The key is sent in an authorization header. It tells OpenRouter which account may make the request; it is not part of the prompt.
4. The request leaves the computer and reaches OpenRouter.
5. OpenRouter routes it to a provider serving the requested model.
6. The provider runs the model and sends a response back through OpenRouter.
7. The SDK turns the response into a Python object such as `hosted_response`.
8. Our code retrieves the returned message text from that object.

The network path is:

```
notebook → openrouter SDK → internet → OpenRouter → model provider
notebook ← Python response object ← internet ← OpenRouter ← model provider
```

The official OpenRouter SDK is a thin client for its REST API: it handles authentication, validates request fields and provides typed response objects. See the [OpenRouter Python SDK](#) and [provider-routing documentation](#).

5.4 What Ollama is

Ollama is an application and local model server. `ollama pull ...` downloads a named model artifact. When Ollama is running, it normally serves an API at `http://localhost:11434/api`. `localhost` means this same computer; it is not a website elsewhere on the internet.

The Python package named `ollama` is the client. It sends a request to the Ollama server. The server loads the requested model onto the available CPU/GPU and memory, runs inference and returns a response. No OpenRouter key is needed for this local API. Model download is a separate earlier network operation.

5.4.1 What happens during an Ollama call

1. Python stores the same prompt and messages in memory.
2. The `ollama` package turns those Python objects into an HTTP request.
3. The request goes to the Ollama server at `localhost`, normally on port 11434.
4. Ollama finds the downloaded model named by `LOCAL_MODEL`.
5. Ollama loads or reuses the model and performs inference on the computer.
6. Ollama returns JSON to the Python client.
7. The client gives us a Python response object such as `local_response`.
8. Our code retrieves the returned message text from that object.

The local path is:

```
notebook → ollama Python client → localhost:11434 → downloaded model
notebook ← Python response object ← localhost:11434 ← downloaded model
```

The local server normally requires no authentication because it listens on the same machine. Do not expose it to a network for this course. See Ollama's official [API introduction](#), [chat endpoint](#) and [Python client](#).

5.5 What is the same, and what differs?

Part of the routine	OpenRouter	Ollama
Research text	same	same
Python <code>messages</code> list	same broad structure	same broad structure

Part of the routine	OpenRouter	Ollama
Client call	<code>client.chat.send(...)</code>	<code>ollama.chat(...)</code>
Destination	remote hosted API	local API at <code>localhost</code>
Authentication	OpenRouter key	none for the local API
Model identifier	hosted model slug	downloaded model tag
Where inference occurs	external provider hardware	local CPU/GPU and memory
Return shape	response with <code>choices[0].message.content</code>	response with <code>message.content</code>
Main constraints	internet, account limits, cost and provider availability	disk, memory, compute speed and installed artifact

Using the same research prompt through both routes holds the requested task relatively stable. It does not make the underlying models equivalent.

5.6 What is actually sent?

An API request has a destination, headers and a body. The SDK handles the HTTP details, but the body is built from values we can inspect. A simplified hosted request body looks like this:

```
import json

hosted_request_body = {
    "model": "openai/gpt-5.2",
    "messages": [
        {"role": "user", "content": "Score this statement from 0 to 2."}
    ],
    "temperature": 0,
}

print(json.dumps(hosted_request_body, indent=2))
```

Python first holds a dictionary containing a string, a list, another dictionary and an integer. The SDK serializes that structure as JSON and sends it to the API. JSON is a text format for exchanging structured data; it is not the model itself.

The key belongs in the authorization header rather than in this body. That is why it should never be concatenated into the prompt or saved in the research material. The response travels back as JSON, and the SDK converts it into a typed response object.

The Ollama request has the same broad pieces but a local destination and route-specific options:

```
local_request_body = {
    "model": "gemma4:e2b-it-qat",
    "messages": hosted_request_body["messages"],
    "options": {"temperature": 0},
}

print(json.dumps(local_request_body, indent=2))
```

The model string, message list and settings are data supplied to the server. `client.chat.send(...)` and `ollama.chat(...)` are Python calls that perform the transport and return an object. Keeping these layers separate makes both debugging and methodological reporting easier.

5.7 Complete this before Week 1

Follow the repository's [platform-specific setup guide](#). In outline:

1. install uv;
2. clone the complete repository or download and extract its ZIP;
3. open a terminal in that folder and run `uv sync --frozen`;
4. install Ollama;
5. run `ollama pull gemma4:e2b-it-qat`;
6. receive the OpenRouter key privately;
7. run `uv run python scripts/preflight_models.py --live`;
8. or complete the [dual-route preflight notebook](#).

The preflight should show successful OpenRouter and Ollama calls without displaying the key. If the local model cannot run on your machine, complete those initial calls on a paired or instructor machine. Hardware ownership does not affect the mark.

Do not download a weekly `.ipynb` file by itself for local work. The notebooks also use the course configuration and, in some weeks, supplied images or data. The [setup guide](#) gives copyable Git and ZIP instructions for macOS, Windows and Linux.

5.8 Starting JupyterLab

Open a terminal in the repository and run:

```
uv run jupyter lab
```

This starts Jupyter with the packages installed in the course environment. Opening a notebook through an unrelated Python installation is a common cause of `No module named ...` errors.

The first notebook cell prints the exact Python executable and course root it found. A healthy local run will show a path inside the repository's `.venv` folder and the full `GenAI_Soc2026` directory. It

then checks the Ollama server and exact course model before the first research call. If either is unavailable, follow the printed **NEXT STEP** and rerun the setup cell.

5.9 Check the Ollama server before opening Python

These terminal commands test three different layers:

```
ollama --version
ollama list
curl http://localhost:11434/api/version
```

- `ollama --version` checks that the command-line application is installed.
- `ollama list` asks the running service which model artifacts are available.
- `curl .../api/version` sends a direct HTTP request to the local API. A small JSON reply means the server is reachable.

If the command exists but the API request fails, Ollama is installed but its server is not currently reachable. If the API works but the course tag is absent from `ollama list`, the server is running but the model still needs to be downloaded.

These are separate installations: the small Python package named `ollama` can import successfully even when the Ollama application is closed or the 4.3 GB course model has not been downloaded.

5.10 Entering the OpenRouter key safely

The instructor supplies one capped key through a private channel. A setup cell asks for it without displaying what you type:

```
import os
from getpass import getpass

if not os.getenv("OPENROUTER_API_KEY"):
    os.environ["OPENROUTER_API_KEY"] = getpass("OpenRouter course key (hidden): ")

print(bool(os.getenv("OPENROUTER_API_KEY")))
```

The final line prints only **True** or **False**. Never paste the key into a saved string, print it, read it aloud, or include its entry cell in a recording.

If a live request says **User not found**, the SDK itself is working but the key is invalid, expired or no longer associated with an active OpenRouter account. Re-enter it once. If the message persists, ask the instructor for a replacement course key rather than reinstalling Python.

5.11 The same research request through both routes

Both calls begin with the same Python list:

```
text = "Most people can be trusted."
instruction = "Score the statement for generalized social trust from 0 to 2. Return one integer."
prompt = instruction + "\n\nStatement: " + text
messages = [{"role": "user", "content": prompt}]
```

`text`, `instruction` and `prompt` are strings. `messages` is a list containing one dictionary. The list is the input supplied to both SDKs.

The list and dictionary are not special LLM syntax. They are ordinary Python objects arranged in the form expected by the APIs. This is where the course begins teaching Python: first we construct a real model request; after the two calls, we inspect the objects we just used.

5.11.1 OpenRouter

```
from openrouter import OpenRouter

HOSTED_MODEL = "openai/gpt-5.2"

with OpenRouter(api_key=os.environ["OPENROUTER_API_KEY"]) as client:
    hosted_response = client.chat.send(
        model=HOSTED_MODEL,
        messages=messages,
        temperature=0,
    )

hosted_choice = hosted_response.choices[0]
hosted_message = hosted_choice.message
hosted_answer = hosted_message.content.strip()
print(hosted_answer)
```

The call sends `messages` to a hosted model. The three following assignments unpack the first returned choice, its message and its text. They are kept separate in Weeks 1–2 so the response path is visible.

5.11.2 Ollama

```
import ollama

LOCAL_MODEL = "gemma4:e2b-it-qat"
```

```

local_response = ollama.chat(
    think=False,
    model=LOCAL_MODEL,
    messages=messages,
    options={"temperature": 0},
)

local_message = local_response.message
local_answer = local_message.content.strip()
print(local_answer)

```

Ollama receives the same message list, but inference runs on the local server. `think=False` asks the course model to return the requested answer directly; otherwise a short output allowance can be consumed by a separate reasoning field before visible answer text is produced. Its generation settings are placed inside `options`, and its response text is reached through `response.message.content`.

5.12 Use the model routine to learn Python objects

After both calls have run, Week 1 pauses to inspect what Python now holds:

```

first_message = messages[0]
routes_match = hosted_answer == local_answer
hosted_character_count = len(hosted_answer)

print(type(text))           # str
print(type(messages))       # list
print(type(first_message))  # dict
print(type(hosted_character_count)) # int
print(type(routes_match))   # bool

```

These types have immediate roles in the model routine:

- a **string** stores text, including the prompt, model name and returned answer;
- a **list** stores ordered items, such as conversation messages or returned choices;
- a **dictionary** stores named fields, such as `role` and `content`;
- an **integer** stores a whole-number quantity, such as a character count;
- a **Boolean** stores `True` or `False`, such as whether two returned strings match.

The SDK also returns objects defined by its own package. We often retrieve their fields with dot notation:

```

hosted_choices = hosted_response.choices
hosted_message = hosted_choices[0].message
hosted_answer = hosted_message.content

```

```
local_message = local_response.message
local_answer = local_message.content
```

Compare three ways of retrieving something:

```
messages[0]           # position 0 in a list
first_message["content"] # field named "content" in a dictionary
hosted_message.content # attribute named content on an SDK object
```

The square brackets do two different jobs because the object on their left differs. A list uses a numeric position. A dictionary uses a key. Dot notation retrieves an attribute supplied by the SDK's response class. We inspect the object with `type(...)` before assuming how to access it.

5.13 Choosing a route from Week 3

```
ROUTE = "ollama" # change to "openrouter" if required
```

Until functions are introduced in Week 9, notebooks show both branches explicitly:

```
if ROUTE == "openrouter":
    # the complete OpenRouter call appears here
    raw_output = hosted_response.choices[0].message.content
else:
    # the complete Ollama call appears here
    raw_output = local_response.message.content
```

This repeats a few lines, but it lets you see which SDK is doing what. The course does not hide the model call inside a helper before you can explain it.

5.14 Cached outputs are contingencies

The repository keeps one clearly labelled teaching return for each route and week in [data/dual_route_recorded_outputs.json](#). These are instructor-authored or previously recorded contingencies, not claimed current model outputs. The instructor supplies the relevant string when a live route fails; it occupies the same point in the routine:

```
raw_output = cached_raw_output
```

If you use it, name that substitution in the recording. A cache lets the Python and validation work continue; it is not evidence about what a live model would return now.

5.15 Safety and access rules

- Use only synthetic, public or instructor-authored material with the shared key.
- Do not expose Ollama outside `localhost` for this course.
- Record the route, requested model, prompt/messages, settings, raw output and date.
- Local operation does not automatically establish privacy, quality or reproducibility.
- The instructor will retest the configured models, budget and provider access two weeks before term.

6 AI use: permitted with responsibility and disclosure

There are a large number of competing policies when it comes to AI. Here is just a selection of several hundred from colleges in the United States: [AI policies across universities](#). You will find that they all differ—sometimes drastically. This is a reasonable, though selective, representation of where we stand at this moment in academia. Some will say you cannot use AI; others will say you cannot do without it. I am sure I would agree with elements of what both sides are saying. In fact, that is basically where I stand.

I will flesh this out with you in Week 1, but here is the thing: you are graduate students at one of the top Sociology programmes in the world. To excel in this field, you will need to understand the field. You will need to think hard—sometimes until your head is spinning. You will need to work hard—it is only this way that you really *understand* something. Or you could rely 100% of the time on some commercial language model to do the hard work for you. If you do that, you will notice that the muscles you require to operate in this world fail to develop. You will notice that others around you are able to speak about the social world and research design with a fluency you have not developed.

I am not here to discipline and punish. I am not interested in doing so. I think language models and modern-day AI software are some of the most exciting technologies around. We can use them to conduct research at a pace and scale previously not possible. But we can also use them as a substitute for thinking. So ask yourself: am I using this as a *tool* to augment my learning and enhance my research? Or am I using this as a *substitute* for myself? Only you can answer that question.

Use of generative AI is permitted in this course, with responsibility and disclosure. If AI materially assists submitted work, include a brief note naming the tool or model, explaining what you used it for, what you incorporated, and how you checked it. When an LLM is itself part of the research method, prompts, outputs, settings, dates, provider or local runtime, and validation decisions belong in the research record. You remain responsible for every claim and every line of code you submit.

Finally, the assignments for this course are principally based on class work and explanation. We will go through notebooks together, check one another's work, and ask you to explain code in full. The individual oral code walkthrough requires you to explain the code yourself without live AI assistance. You may use AI to help you learn, debug, and develop your work, but you must still understand what the work does—so do not outsource your learning.

Sermon over. Let's get to work!

7 Assessment and marking

Component	Weight
Weekly completion tasks: ten best submissions	20%
Midterm project proposal	20%
Individual oral code walkthrough	5%
Final-project milestones	10%
Final research-design package	25%
Final presentation	10%
Participation and reading preparation	10%

7.1 Weekly completion tasks

Ten best submissions from Weeks 1–13 count. Week 12 is deliberately low-stakes and may replace a missed task. Each narrated screen recording is due by **5:00 p.m. Eastern on the Tuesday before the next class**. Use the [recording instructions](#) and [Box upload email](#).

In the recording, students run the complete block supplied on the week’s Python page, explain the input and output of every operation, change one named input or parameter, and explain what the model return means for the week’s methodological question. From Week 2, the normal route is a live call through OpenRouter or a supported local model; a supplied cached response may be used when access fails. Students are not graded for editing quality, code elegance or efficient abstraction.

A task is complete when the recording makes the student’s understanding audible: the relevant Python values and types are identified, the model call and returned object are explained, the requested change is shown, one limit is stated and the connection to a reading is made. Weekly tasks do not receive numerical quality grades. Missing elements may be supplied within one week.

7.2 Midterm project proposal

The proposal is released after Week 6 and submitted before Week 7. In 1,200–1,500 words, you set out a sociological question, locate it in a focused body of literature, specify the evidence and research design, and explain exactly where an LLM would enter the study. Include one concrete input → operation → expected output → researcher check, along with a validation plan, likely failures, ethical issues, the records you would preserve and the next steps toward the final project.

This is the first substantial version of the final project. It is not a Python test, and it does not require a completed empirical study or an exhaustive literature review.

7.3 Individual oral code walkthrough

Shortly afterwards, each student completes a separate 10–12 minute individual walkthrough. With a short extract from a weekly task or suitable proposal example visible, you identify the inputs and Python types, explain every line or small block, predict the output and one modification, and connect the computation to its sociological meaning. This is an understanding assessment, not a syntax quiz. No live generative assistance is used during it.

7.4 Final project

The final project is a self-designed, LLM-related sociological research design and class presentation. A minimal input/output trace or proof of concept is expected where feasible, but you are not required to complete a full empirical paper or obtain definitive findings.

After receiving feedback on the proposal, submit a short revision note on 28 October explaining what you will change. The 11 November design clinic tests the revised validation plan and minimal trace; the 25 November studio produces a recoverable discrepancy or progress record. The project presentation takes place on 9 December. During the examination period, submit the final package: sociological motivation, inferential target, data and sampling plan, proposed LLM workflow, prototype where feasible, validation strategy, failure modes and ethics, reproducibility record, AI-use disclosure, and a brief account of how the proposal changed.

The final package is assessed for sociological importance, fit between claim and design, transparent implementation, validation, substantive interpretation, reproducibility and communication. Code sophistication is not an independent virtue.

7.5 Participation and reading preparation

Participation can take the form of a reading question, explanation of a code block, validation critique, project feedback or a short written contribution. Speaking frequency alone does not determine the mark.

Part I

Evidence, measurement and interpretation

8 Week 1 — What kind of research technology is an LLM?

8.1 This week's sociological question

When does a model response become part of a defensible research process rather than a piece of plausible text?

8.2 Connection to the preceding material

This is the starting point for the semester. We begin with a familiar action—asking a model to do something—and slow it down. The rest of the course depends on being able to distinguish the request, the model operation, the returned object, the researcher's transformation and the later claim.

8.3 Learning outcomes

By the end of the week, you should be able to distinguish a chat interface, an API, a routing service and a local runtime; identify strings, lists and dictionaries in a model request; explain why prompts, settings and raw outputs belong in the research record; and state what a saved record does and does not establish.

8.4 Essential readings and what each contributes

- Barrie, Argyle, Bisbee, Heseltine, Lucas, Mellon, Palmer, Roberts and Spirling (2026), “AI and Research Methods,” introduction and §10.7; PDF pp. 1–2 and 34–35. The introduction names the chapter's six LLM applications. Section 10.7 explains why exact replay and replication of a substantive claim are different standards, then specifies the records every LLM workflow should preserve. The chapter appears on pp. 313–364 of the edited volume. [Download the chapter PDF](#) · [APSA Preprints record](#). (Barrie et al. 2026)
- Davidson and Karell (2025), “Integrating Generative Artificial Intelligence into Social Science Research,” *Sociological Methods & Research*. A framework organized around measurement, prompting and simulation. [DOI](#). (Davidson and Karell 2025)
- Alvero, Stoltz, Stuhler and Taylor (2026), “Generative AI in Sociological Research,” *Sociological Science*. A discipline-specific account of where generative AI is appearing and which judgments cannot be delegated. [Article](#). (Alvero et al. 2026)

8.5 Additional readings grouped by purpose

- **For a concrete reporting standard:** Feuerriegel et al. (2026), “A Reporting Checklist for Large Language Models in Behavioural Science,” *Nature Human Behaviour*. DOI. ([Feuerriegel et al. 2026](#))
- **For the replication problem:** Barrie, Palmer and Spirling (2025), “Replication for Language Models.” [Paper](#). ([Barrie, Palmer, and Spirling 2025](#))

8.6 Sociological and methodological background

Social scientists have always relied on research technologies: questionnaires, recording devices, codebooks, statistical packages and archives. Each makes some work easier while standardizing or hiding other decisions. LLMs extend that pattern. They can turn a natural-language instruction into a classification, summary, interview probe or simulated response, but the fluency of the return makes it unusually easy to forget the chain of decisions that produced it.

Reproducibility is therefore introduced at the beginning, not saved for the end. A model name may point to a changing service; a provider may route the same request to different infrastructure; a prompt may be edited without versioning; and a polished answer may be copied without its raw response. The Week 1 question is modest: can another researcher see what was asked, what answered and what you did next?

8.7 Synthesis of the current LLM literature

The three essential readings make different kinds of claim. Davidson and Karell offer a broad methodological framework. Alvero and colleagues ask what generative AI means for sociology as a discipline. The course chapter inventories specific operations and argues that familiar standards must be reconsidered when the transformation is prompt-sensitive and the model is proprietary. Messeri and Crockett’s broader warning about an “illusion of understanding” is relevant here: faster production can encourage broader claims without improving the evidence underneath them. The practical response is not to reject the technology. It is to make the operation visible and keep validation separate from generation.

8.8 Slides

The slides are available in the [online Week 1 chapter](#).

[Continue to the Week 1 Python work](#).

8.9 Summary and preparation for next week

An inspectable record does not validate a model answer. It preserves the objects needed to ask a better question about validity. Next week, we use that distinction when a model's answer becomes a label in a dataset.

9 Week 1 Python — First calls and Python objects

[Return to the Week 1 readings and slides.](#)

9.1 Python walkthrough: make the first two model calls

Today's routine: One statement → one shared prompt → two model calls → two research records.

Python introduced: notebook cells, strings, variables, functions we call, lists, dictionaries, indexing, SDK objects, attributes, methods, integers and Booleans.

Your change: replace the trust statement and run both routes again.

The routine begins with the model input. It does not postpone the live call until after unrelated Python exercises. Predict what each block will create before running it.

9.1.1 Load the course settings and SDKs

Run the notebook's **Prepare the notebook environment** cell first. In Colab, that cell installs the two Python SDKs and sets `IN_COLAB = True`. Locally, it checks that Jupyter was started from the course environment; if packages are missing, close Jupyter and run `uv sync` followed by `uv run jupyter lab` from the repository root.

This next cell reads the two centrally recorded model names and asks for the hosted key without displaying it. Treat the key entry as setup; never include it in the recording.

```
import json
import os
from getpass import getpass
from pathlib import Path

try:
    import ollama
    from openrouter import OpenRouter
except ModuleNotFoundError as error:
    raise ModuleNotFoundError(
        "A course SDK is missing. Open a terminal in the complete GenAI_Soc2026 "
        "folder, run 'uv sync --frozen', then run this task with 'uv run python'."
    ) from error
```

```

ROOT = Path.cwd()
while not (ROOT / "config" / "course_models.json").exists() and ROOT != ROOT.parent:
    ROOT = ROOT.parent

if not (ROOT / "config" / "course_models.json").exists():
    raise FileNotFoundError(
        "The complete GenAI_Soc2026 repository could not be found. A task or "
        "notebook downloaded by itself is not enough for local work. Open a terminal "
        "in the complete course folder and run this file from there."
    )

config = json.loads((ROOT / "config" / "course_models.json").read_text())
HOSTED_MODEL = config["hosted"]["model"]
LOCAL_MODEL = config["local"]["model"]

if not os.getenv("OPENROUTER_API_KEY"):
    os.environ["OPENROUTER_API_KEY"] = getpass("OpenRouter course key (hidden): ")

print("Hosted model:", HOSTED_MODEL)
print("Local model:", LOCAL_MODEL)

```

i Read this setup cell

Input: installed Python packages, `config/course_models.json`, and—if it is not already set—the hidden OpenRouter key. **Operations:** `import` makes an installed tool available; `Path.cwd()` gives Python the current folder; the `while` block walks upward until it finds the course configuration; `json.loads(...)` turns the file's JSON text into a dictionary; square brackets retrieve the two model names. **Output:** `HOSTED_MODEL` and `LOCAL_MODEL` are strings. `getpass(...)` accepts the key without echoing it. The folder-search code is supplied setup and is not assessed.

9.1.2 Store the research material and construct one shared message list

The research material begins as strings. The list named `messages` is the exact shared input to both SDKs.

```

text = "Most people can be trusted."
instruction = (
    "Score the statement for generalized social trust from 0 to 2. "
    "Return one integer."
)
prompt = instruction + "\n\nStatement: " + text
messages = [{"role": "user", "content": prompt}]

print("text:", text)

```

```
print("type(text):", type(text))
print("messages:", messages)
print("type(messages):", type(messages))
```

i Input and output

`text` and `instruction` are strings. `+` joins those strings with the labelled statement to create `prompt`; `\n` means start a new line. The braces create one dictionary with two named fields, and the brackets place that dictionary in a list called `messages`. Both clients will receive this same list. `type(...)` reports the kind of Python object without changing it.

9.1.3 Send the message list through OpenRouter and unpack its return

Inference occurs remotely. The response path is unpacked one object at a time before the returned text is printed.

```
with OpenRouter(api_key=os.environ["OPENROUTER_API_KEY"]) as client:
    hosted_response = client.chat.send(
        model=HOSTED_MODEL,
        messages=messages,
        temperature=0,
    )

hosted_choice = hosted_response.choices[0]
hosted_message = hosted_choice.message
hosted_answer = hosted_message.content.strip()
print("OpenRouter raw text:", hosted_answer)
```

i What the hosted call does

The call receives three named inputs: the hosted model string, the message list and `temperature=0`. `with ... as client` opens the SDK client and closes it after the indented call. The service returns a response object containing a list named `choices`; `[0]` selects the first choice. `.message` retrieves its message and `.content` retrieves the returned text. `.strip()` removes surrounding whitespace but does not change the score itself.

9.1.4 Send the same message list through Ollama and unpack its return

Inference occurs through the Ollama server on this computer. The same messages enter, but the call and response path differ.

```
if IN_COLAB:
    local_response = None
    local_message = None
```

```

local_answer = None
print("Ollama was not called: Colab cannot reach the local Ollama server.")
else:
    local_response = ollama.chat(think=False,
                                model=LOCAL_MODEL,
                                messages=messages,
                                options={"temperature": 0},
                                )
    local_message = local_response.message
    local_answer = local_message.content.strip()
    print("Ollama raw text:", local_answer)

```

i What the local call does

if `IN_COLAB` checks the Boolean created by the setup cell. In Colab, the three local values become `None`, Python's marker for "no value," and no local request is attempted. Locally, `ollama.chat(think=False, ...)` sends the same message list to the Ollama server on this computer. Ollama's response shape differs: the message is available as `.message`, without a `choices` list. A local route changes where inference occurs; it does not validate the score.

9.1.5 Use the request to learn strings, lists and dictionaries

Both calls have now run. We can use the objects involved in those calls to learn Python rather than switching to an unrelated example.

```

first_message = messages[0]

print("text value:", text)
print("text type:", type(text))
print("messages value:", messages)
print("messages type:", type(messages))
print("number of messages:", len(messages))
print("first message:", first_message)
print("first message type:", type(first_message))
print("role field:", first_message["role"])
print("content field:", first_message["content"])

```

i Read the objects from left to right

`text` is a string: one text value. `messages` is a list: an ordered collection whose first position is numbered 0. `messages[0]` therefore retrieves the first item and = assigns it to the new variable `first_message`. That item is a dictionary: a collection of named fields. `first_message["role"]` retrieves the value stored under the key "role"; `first_message["content"]` retrieves the actual prompt. `len(messages)` calls a built-in function and returns the integer 1. Parentheses contain the value passed to a function; com-

mas separate multiple inputs. We call `print`, `type` and `len`, but students do not define a function until Week 9. None of these operations contacts a model. They inspect the input already held by Python.

9.1.6 Use the returns to learn objects, attributes and methods

The two SDKs returned objects defined by their packages. They are not plain strings, lists or dictionaries, even though they contain some of those types.

```
hosted_choices = hosted_response.choices

print("hosted response type:", type(hosted_response))
print("hosted choices type:", type(hosted_choices))
print("first hosted choice type:", type(hosted_choice))
print("hosted message type:", type(hosted_message))
print("hosted answer type:", type(hosted_answer))

print("local response type:", type(local_response))
print("local message type:", type(local_message))
print("local answer type:", type(local_answer))
```

i Why some fields use dots and others use brackets

`hosted_response.choices` uses dot notation to retrieve an attribute supplied by the Open-Router response class. Because `choices` is a list, `[0]` then retrieves its first item. `.message` and `.content` retrieve further attributes. By contrast, `first_message["content"]` uses a dictionary key. Finally, `.strip()` is a string method: it returns a new string without surrounding whitespace. A useful beginner question is therefore not simply “what does this line do?” but “what type of object is on the left of the dot or brackets?”

9.1.7 Derive an integer and a Boolean from the returned strings

```
hosted_character_count = len(hosted_answer)
local_character_count = None if local_answer is None else len(local_answer)
routes_match = None if local_answer is None else hosted_answer == local_answer

print("hosted character count:", hosted_character_count)
print("count type:", type(hosted_character_count))
print("routes returned identical text:", routes_match)
print("comparison type:", type(routes_match))
```


i Two more built-in Python types

`len(hosted_answer)` counts the characters in a returned string and produces an integer. The short `if ... else ...` expression first checks whether an Ollama result exists. In Colab it returns `None`; locally, `==` compares the two strings and produces the Boolean `True` or `False`. `None` here means “not run,” not model disagreement. Identical returned strings would show agreement on this run, but would not show that either answer measures social trust validly.

9.1.8 Preserve the two runs as separate research records

Each dictionary keeps route, requested model, input and raw output together. This makes the comparison inspectable.

```
hosted_record = {
    "route": "openrouter",
    "model": HOSTED_MODEL,
    "input": text,
    "raw_output": hosted_answer,
}
local_record = {
    "route": "ollama",
    "model": LOCAL_MODEL,
    "input": text,
    "raw_output": local_answer,
}

research_records = [hosted_record, local_record]

print(hosted_record)
print(local_record)
print("record collection type:", type(research_records))
print("first recorded route:", research_records[0]["route"])

# ONE CHANGE: replace text with "You cannot be too careful in dealing with people."
# Then rerun every cell from the shared message list onward.
```

i Why these dictionaries matter

Each pair of braces creates a dictionary. Text before a colon is a field name; the value after it is what the record stores. The brackets around the two records create a list. `research_records[0]["route"]` first selects item 0 from that list and then selects the "route" field from the chosen dictionary. The two outputs are not averaged or declared correct. They remain attached to route, requested model and input so a later reviewer can see what produced each proposed score.

9.1.9 Methodological check

The two scores are proposed measurements. Neither route establishes that the score matches the intended construct or a defensible human codebook. If `local_answer` is `None`, the local route was deliberately skipped in Colab; it is not a model response.

9.2 Workbook and environment links

[Open the Week 1 notebook in Google Colab](#) · [Download the notebook](#) · [Download the Python exercise as a .py file](#) · [Download the runnable offline bundle](#)

This page, the notebook and the .py download contain the same assessed routine. Use the notebook for the guided cell-by-cell walkthrough and the script when you want to run the complete routine from the repository root.

9.3 Take-home exercise

Estimated work and recording time: 30–45 minutes.

Run both routes on the original statement and on `You cannot be too careful in dealing with people`. In the narrated recording, explain every string, the message list, where each inference runs, each returned object and each research record. Finish by explaining why either score still needs validation.

Submit one continuous screen recording with audio. Show the original run and the named change. Explain every operation's input, Python type and output; identify the access route; connect the methodological limit to one essential reading. Upload it using the [recording and submission instructions](#) by **5:00 p.m. Eastern on the Tuesday before the next class**. Never show or read the shared key. If a cached contingency was necessary, identify it.

10 Week 2 — When can an LLM label count as a measure?

10.1 This week's sociological question

What evidence do we need before an LLM label can function as a sociological measure?

10.2 Connection to the preceding material

Week 1 preserved a model request and response. This week the response enters a dataset. That raises a stronger question: does the label measure the construct we intended, and what happens to the substantive estimate when it is wrong?

10.3 Learning outcomes

You should be able to define a construct, unit and boundary case; explain how a codebook becomes a promptbook; distinguish prompt stability from validity; calculate confusion counts, precision and recall; and trace label error into a prevalence estimate.

10.4 Essential readings and what each contributes

- Barrie et al. (2026), “AI and Research Methods,” §§10.1.1–10.1.5. Introduces annotation, its methodological lineage, model and prompt choices, validation, and downstream inference. [Download the chapter PDF](#) · [APSA Preprints record](#). (Barrie et al. 2026)
- Stuhler, Dang Ton and Ollion (2025), “From Codebooks to Promptbooks,” *Sociological Methods & Research* 54(3): 794–848. Connects established coding practice to explicit prompt instructions and validation. [DOI](#). (Stuhler, Dang Ton, and Ollion 2025)
- Barrie, Palaiologou and Törnberg (2026), “Prompt Stability Scoring for Text Annotation with Large Language Models.” Treats disagreement across semantically similar prompts as a reliability diagnostic. [DOI](#). (Barrie, Palaiologou, and Törnberg 2026)

10.5 Additional readings grouped by purpose

- **For alternative classification strategies:** Chae and Davidson (2026), “Large Language Models for Text Classification,” *Sociological Methods & Research*. DOI. ([Chae and Davidson 2026](#))
- **For the broader measurement argument:** Davidson and Karell (2025). DOI. ([Davidson and Karell 2025](#))

10.6 Sociological and methodological background

Text does not arrive as a variable. Sociologists and political scientists have long moved from documents to data by defining constructs, writing codebooks, training research assistants and adjudicating hard cases. Computational text analysis made parts of this work scalable; crowdworker platforms such as MTurk and Prolific made it easier to distribute judgments beyond expert teams. Those changes lowered cost but did not remove the need to decide what counts as evidence for the construct.

An LLM prompt is therefore not a free-standing instruction. It contains a codebook in compressed form: the label names, inclusion and exclusion rules, examples, output format and sometimes a role or rationale. Stuhler and colleagues call attention to this move from codebook to promptbook. The useful comparison is not “human versus AI” in the abstract, but two complete measurement procedures applied to a stated use.

10.7 Synthesis of the current LLM literature

Three issues should remain separate. **Reliability** asks whether reasonable repetitions or prompt formulations agree. **Validity** asks whether the labels represent the intended construct. **Downstream error** asks how mistakes affect the estimate or model in which the labels are used. Prompt Stability Score addresses the first. A human-coded validation sample and difficult boundary cases contribute to the second. Confusion counts and design-based corrections address the third. A classifier can have high overall accuracy and still inflate a rare category if its false positives are concentrated in one direction.

10.8 Slides

The slides are available in the [online Week 2 chapter](#).

[Continue to the Week 2 Python work](#).

10.9 Summary and preparation for next week

Stability tells us where a measurement procedure changes under reasonable prompt variation; independent evidence tells us whether it is right; downstream analysis tells us why the distinction matters. Week 3 moves from assigning predefined labels to developing qualitative interpretations.

11 Week 2 Python — LLM annotation

[Return to the Week 2 readings and slides.](#)

11.1 Python walkthrough: use an llm as an annotator

Today's routine: One housing comment → one codebook prompt → two model labels → three comparisons.

Python introduced: lists, dictionaries, indexing, dictionary keys and equality comparisons.

Your change: replace the conditional comment with an opposition comment and rerun both routes.

The routine begins with the model input. It does not postpone the live call until after unrelated Python exercises. Predict what each block will create before running it.

11.1.1 Load the course settings and SDKs

Run the notebook's **Prepare the notebook environment** cell first. In Colab, that cell installs the two Python SDKs and sets `IN_COLAB = True`. Locally, it checks that Jupyter was started from the course environment; if packages are missing, close Jupyter and run `uv sync` followed by `uv run jupyter lab` from the repository root.

This next cell reads the two centrally recorded model names and asks for the hosted key without displaying it. Treat the key entry as setup; never include it in the recording.

```
import json
import os
from getpass import getpass
from pathlib import Path

try:
    import ollama
    from openrouter import OpenRouter
except ModuleNotFoundError as error:
    raise ModuleNotFoundError(
        "A course SDK is missing. Open a terminal in the complete GenAI_Soc2026 "
        "folder, run 'uv sync --frozen', then run this task with 'uv run python'."
    ) from error

ROOT = Path.cwd()
```

```

while not (ROOT / "config" / "course_models.json").exists() and ROOT != ROOT.parent:
    ROOT = ROOT.parent

if not (ROOT / "config" / "course_models.json").exists():
    raise FileNotFoundError(
        "The complete GenAI_Soc2026 repository could not be found. A task or "
        "notebook downloaded by itself is not enough for local work. Open a terminal "
        "in the complete course folder and run this file from there."
    )

config = json.loads((ROOT / "config" / "course_models.json").read_text())
HOSTED_MODEL = config["hosted"]["model"]
LOCAL_MODEL = config["local"]["model"]

if not os.getenv("OPENROUTER_API_KEY"):
    os.environ["OPENROUTER_API_KEY"] = getpass("OpenRouter course key (hidden): ")

print("Hosted model:", HOSTED_MODEL)
print("Local model:", LOCAL_MODEL)

```

i Read this setup cell

Input: installed Python packages, `config/course_models.json`, and—if it is not already set—the hidden OpenRouter key. **Operations:** `import` makes an installed tool available; `Path.cwd()` gives Python the current folder; the `while` block walks upward until it finds the course configuration; `json.loads(...)` turns the file's JSON text into a dictionary; square brackets retrieve the two model names. **Output:** `HOSTED_MODEL` and `LOCAL_MODEL` are strings. `getpass(...)` accepts the key without echoing it. The folder-search code is supplied setup and is not assessed.

11.1.2 Store one comment, the codebook and a human reference label

Run this block, print its output and explain what enters and leaves the operation. The next block uses the value created here.

```

comment = "I support the plan if rents remain affordable."
codebook = {
    "SUPPORT": "unconditional support for the proposal",
    "OPPOSE": "unconditional opposition to the proposal",
    "UNCLEAR": "conditional, mixed, procedural, or insufficient evidence",
}
human_label = "UNCLEAR"
print(comment)
print(codebook["UNCLEAR"])

```

i Three different inputs

`comment` is the text to classify. `codebook` is a dictionary mapping each permitted label to its definition. `human_label` is a separate reference judgment; it is not sent to the model. `codebook["UNCLEAR"]` uses a key to retrieve one definition and demonstrates how dictionaries differ from lists.

11.1.3 Construct the exact message list used by both routes

Run this block, print its output and explain what enters and leaves the operation. The next block uses the value created here.

```
prompt = (
    "Apply this codebook: " + json.dumps(codebook) +
    "\nReturn only SUPPORT, OPPOSE, or UNCLEAR.\nComment: " + comment
)
messages = [{"role": "user", "content": prompt}]
print(messages[0]["content"])
```

i From codebook to model input

`json.dumps(codebook)` turns the Python dictionary into readable text that can be placed inside the prompt. The `+` signs join codebook, instruction and comment. The resulting `messages` list contains one user message. `[0]` retrieves the first list item; `["content"]` then retrieves that dictionary's content field.

11.1.4 Make and unpack the OpenRouter call

Run this block, print its output and explain what enters and leaves the operation. The next block uses the value created here.

```
with OpenRouter(api_key=os.environ["OPENROUTER_API_KEY"]) as client:
    hosted_response = client.chat.send(
        model=HOSTED_MODEL, messages=messages, temperature=0,
    )
hosted_choice = hosted_response.choices[0]
hosted_raw = hosted_choice.message.content
print("OpenRouter raw return:", hosted_raw)
hosted_label = hosted_raw.strip().upper()
print("OpenRouter label:", hosted_label)
```


Hosted label

The inputs are the hosted model, the shared message list and zero temperature. [0] chooses the first returned choice, `.message.content` retrieves its text, `.strip()` removes spare whitespace and `.upper()` standardizes letter case. The final output, `hosted_label`, remains a string and may still fall outside the codebook.

11.1.5 Make and unpack the Ollama call

Run this block, print its output and explain what enters and leaves the operation. The next block uses the value created here.

```
if IN_COLAB:
    local_response = None
    local_raw = None
    local_label = None
    print("Ollama was not called: Colab cannot reach the local Ollama server.")
else:
    local_response = ollama.chat(think=False,
                                model=LOCAL_MODEL, messages=messages, options={"temperature": 0},
                                )
    local_raw = local_response.message.content
    print("Ollama raw return:", local_raw)
    local_label = local_raw.strip().upper()
    print("Ollama label:", local_label)
```

Local label

if `IN_COLAB` checks the Boolean created by the setup cell. In Colab, no local request is attempted and the local values become `None`. Locally, the same messages enter Ollama. Its temperature sits inside an `options` dictionary and its returned text is found at `local_response.message.content`. Standardizing case makes the two strings easier to compare; it does not repair a substantively wrong label.

11.1.6 Compare route agreement and human-reference agreement

Run this block, print its output and explain what enters and leaves the operation. The next block uses the value created here.

```
routes_agree = None if local_label is None else hosted_label == local_label
hosted_matches_human = hosted_label == human_label
local_matches_human = None if local_label is None else local_label == human_label
print("Routes agree:", routes_agree)
print("OpenRouter matches human label:", hosted_matches_human)
print("Ollama matches human label:", local_matches_human)
```

```
# ONE CHANGE: replace comment with
# "I oppose the rezoning proposal because it will displace tenants."
# Then rerun from the message-construction cell.
```

i Three Boolean outputs

Each `==` asks whether two strings are equal and returns `True` or `False`. The short `if ... else ...` expressions first check whether Ollama ran. In Colab the local comparisons return `None`, which means “not run,” not disagreement. `routes_agree` compares the two models; the other two values compare each model with the independently supplied human reference.

11.1.7 Methodological check

Agreement between the routes is a reproducibility observation. Agreement with one human label is criterion evidence. Neither alone proves that the codebook measures the construct well. If an Ollama comparison prints `None`, the local route was deliberately skipped in Colab.

11.2 Workbook and environment links

[Open the Week 2 notebook in Google Colab](#) · [Download the notebook](#) · [Download the Python exercise as a .py file](#) · [Download the runnable offline bundle](#)

This page, the notebook and the `.py` download contain the same assessed routine. Use the notebook for the guided cell-by-cell walkthrough and the script when you want to run the complete routine from the repository root.

11.3 Take-home exercise

Estimated work and recording time: 30–45 minutes.

Run both routes on the changed opposition comment. Explain the comment string, codebook dictionary, message list, `[0]`, each response path and all three Boolean comparisons. State whether disagreement is between routes, with the human reference, or both.

Submit one continuous screen recording with audio. Show the original run and the named change. Explain every operation’s input, Python type and output; identify the access route; connect the methodological limit to one essential reading. Upload it using the [recording and submission instructions](#) by **5:00 p.m. Eastern on the Tuesday before the next class**. Never show or read the shared key. If a cached contingency was necessary, identify it.

12 Week 3 — What can an LLM contribute to qualitative interpretation?

12.1 This week's sociological question

Where can a model assist an iterative qualitative analysis without replacing the researcher's engagement with cases, context and counterevidence?

12.2 Connection to the preceding material

Week 2 began with a construct and a predefined label set. Qualitative analysis often begins less settled. Codes develop, comparisons redirect attention and memos connect passages to emerging ideas. The problem is not only whether a model applies a rule correctly, but how its suggestions reorganize the researcher's attention.

12.3 Learning outcomes

You should be able to distinguish retrieval, coding, theming and interpretation; describe familiarization, focused coding, constant comparison and memo-writing; assign an LLM a bounded role; trace a proposed theme to source passages and negative cases; and explain what a source-complete record still cannot establish.

12.4 Essential readings and what each contributes

- **Methods:** Timmermans and Tavory (2012), "Theory Construction in Qualitative Research," *Sociological Theory*. Read the complete article. It explains abductive analysis: surprising observations become analytically useful through comparison with alternative theoretical accounts. DOI. ([Timmermans and Tavory 2012](#))
- **Methods and LLMs:** Than et al. (2025), "Updating 'The Future of Coding'," *Sociological Methods & Research*. Read the complete article. It provides a researcher-LLM-researcher workflow and an empirical comparison of human and model-assisted coding. DOI. ([Than et al. 2025](#))
- **Substantive application:** Viterna (2006), "Pulled, Pushed, and Persuaded," *American Journal of Sociology*. Read the complete article. Viterna uses life histories and event-level comparisons to explain different routes into high-risk mobilization; we use it to see what qualitative analysis must preserve before asking an LLM for themes. DOI. ([Viterna 2006](#))

12.5 Additional readings grouped by purpose

- **For reflexivity about the model interaction:** Ibrahim and Voyer (2026), “Qualitative Research with LLM Chatbots,” *Qualitative Research*. DOI. ([Ibrahim and Voyer 2026](#))
- **For a recent empirical implementation:** Jack, Cooper and Flower (2026), “Automating the Qualitative Interview?” *Methodological Innovations*. DOI. ([Jack, Cooper, and Flower 2026](#))

12.6 Sociological and methodological background

Qualitative analysis is not a single request for “the themes.” Researchers read and re-read material, write provisional codes, compare passages and cases, record hunches, search for exceptions and revise the relationship between data and theory. Timmermans and Tavory emphasize abduction: an unexpected observation becomes productive when it is developed against alternative theoretical accounts. A list of frequent topics is therefore not equivalent to an interpretation.

Viterna shows what this looks like in substantive research. Her explanation of women’s routes into Salvadoran guerrilla organizations depends on reconstructing sequences within life histories and comparing different paths into mobilization. The same broad topic can appear in several accounts while doing different explanatory work. That is exactly the kind of difference a decontextualized list of themes can erase.

Our worked corpus consists of short interview excerpts about tenant mutual-aid networks. Each excerpt has an ID, speaker, question, preceding context and text. The research question asks how participants describe the relationship between mutual aid and formal housing organizations. Giving the listener this setup matters: the same sentence could support a different interpretation in a study of friendship, political mobilization or service provision.

12.7 Synthesis of the current LLM literature

Than and colleagues show that models can contribute useful code suggestions while researchers retain decisions about meaning and iteration. Ibrahim and Voyer ask researchers to document how the chatbot’s categories and conversational affordances enter the analysis. Both point away from one-shot automation. Context windows are large but finite; calls may not preserve cumulative analytic memory; summaries can smooth over contradictions; and a model can cite genuine passages while overstating what they show. The worked exercise therefore asks the model for a provisional suggestion and a source ID, then sends the researcher back to the passage. Viterna gives us the harder standard: can the analysis preserve chronology, comparison across cases and evidence that does not fit the first proposed theme?

12.8 Slides

The slides are available in the [online Week 3 chapter](#).

[Continue to the Week 3 Python work](#).

12.9 Summary and preparation for next week

An LLM can widen the set of possible codes or retrieve passages quickly. The analysis still depends on context, comparison, cumulative memo-writing and the researcher's account of why an interpretation changed. Week 4 asks how the source record itself is produced when the evidence is visual, auditory and sequential.

13 Week 3 Python — Qualitative suggestions in JSON

[Return to the Week 3 readings and slides.](#)

13.1 Python walkthrough: ask for a qualitative suggestion in json

Today's routine: Two excerpts → one structured request → one provisional theme → return to the cited passage.

Python introduced: a route selector, `if/else`, lists of dictionaries, `json.dumps(...)` and `json.loads(...)`.

Your change: replace excerpt e02, rerun one route and check the new citation.

The routine begins with the model input. It does not postpone the live call until after unrelated Python exercises. Predict what each block will create before running it.

13.1.1 Load the course settings and SDKs

Run the notebook's **Prepare the notebook environment** cell first. It installs missing SDKs in Colab, checks the local `uv` environment, adds the repository root to Python's import path and sets `IN_COLAB`. Colab uses OpenRouter because it cannot reach the Ollama server on your computer.

This next cell reads the two centrally recorded model names and asks for the hosted key without displaying it. Treat the key entry as setup; never include it in the recording.

```
import json
import os
from getpass import getpass
from pathlib import Path

try:
    import ollama
    from openrouter import OpenRouter
except ModuleNotFoundError as error:
    raise ModuleNotFoundError(
        "A course SDK is missing. Open a terminal in the complete GenAI_Soc2026 "
        "folder, run 'uv sync --frozen', then run this task with 'uv run python'."
    ) from error
```

```

ROOT = Path.cwd()
while not (ROOT / "config" / "course_models.json").exists() and ROOT != ROOT.parent:
    ROOT = ROOT.parent

if not (ROOT / "config" / "course_models.json").exists():
    raise FileNotFoundError(
        "The complete GenAI_Soc2026 repository could not be found. A task or "
        "notebook downloaded by itself is not enough for local work. Open a terminal "
        "in the complete course folder and run this file from there."
    )

config = json.loads((ROOT / "config" / "course_models.json").read_text())
HOSTED_MODEL = config["hosted"]["model"]
LOCAL_MODEL = config["local"]["model"]

if not os.getenv("OPENROUTER_API_KEY"):
    os.environ["OPENROUTER_API_KEY"] = getpass("OpenRouter course key (hidden): ")

print("Hosted model:", HOSTED_MODEL)
print("Local model:", LOCAL_MODEL)

```

i Read this setup cell

Input: installed Python packages, `config/course_models.json`, and—if it is not already set—the hidden OpenRouter key. **Operations:** `import` makes an installed tool available; `Path.cwd()` gives Python the current folder; the `while` block walks upward until it finds the course configuration; `json.loads(...)` turns the file's JSON text into a dictionary; square brackets retrieve the two model names. **Output:** `HOSTED_MODEL` and `LOCAL_MODEL` are strings. `getpass(...)` accepts the key without echoing it. The folder-search code is supplied setup and is not assessed.

13.1.2 Choose one route and store two identified excerpts

Run this block, print its output and explain what enters and leaves the operation. The next block uses the value created here.

```

ROUTE = "openrouter" if IN_COLAB else "ollama" # Ollama is the local default
excerpts = [
    {"id": "e01", "text": "After several exchanges, I attended the tenants' meeting."},
    {"id": "e02", "text": "I accepted help but did not attend political meetings."},
]
print("Route:", ROUTE)
print("First excerpt:", excerpts[0])

```

i Route and source records

ROUTE is a string controlling which later branch runs. `excerpts` is a list containing two dictionaries; every dictionary keeps an excerpt ID beside its text. Keeping IDs with passages makes it possible to test whether a model's cited evidence actually exists.

13.1.3 Convert the excerpts into prompt text and construct messages

Run this block, print its output and explain what enters and leaves the operation. The next block uses the value created here.

```
prompt = (
    "Suggest one provisional theme. Return JSON with exactly theme, evidence_id, "
    "and question_for_researcher. Excerpts: " + json.dumps(excerpts)
)
messages = [{"role": "user", "content": prompt}]
print(prompt)
```

i Preparing the qualitative input

`json.dumps(excerpts)` converts the list of source dictionaries to text without discarding their IDs. The prompt states the three required output fields. The result is placed in the familiar one-message list. Its output is model input, not a theme or finding.

13.1.4 Make the selected route's call without hiding either branch

Run this block, print its output and explain what enters and leaves the operation. The next block uses the value created here.

```
if ROUTE == "openrouter":
    with OpenRouter(api_key=os.environ["OPENROUTER_API_KEY"]) as client:
        response = client.chat.send(
            model=HOSTED_MODEL,
            messages=messages,
            temperature=0,
            response_format={"type": "json_object"},
        )
    raw_output = response.choices[0].message.content
else:
    response = ollama.chat(think=False,
        model=LOCAL_MODEL,
        messages=messages,
        format="json",
        options={"temperature": 0},
    )
```



```
raw_output = response.message.content

print("Raw JSON text:", raw_output)
```

i Reading the branch

`if ROUTE == "openrouter"` runs only when that comparison is true; otherwise `else` runs the Ollama code. Both complete calls remain visible. Each branch assigns returned text to the same name, `raw_json`, so the parsing step below is identical whichever route was chosen.

13.1.5 Parse the JSON string and return to its cited source

Run this block, print its output and explain what enters and leaves the operation. The next block uses the value created here.

```
suggestion = json.loads(raw_output)
evidence_id = suggestion["evidence_id"]
cited_excerpt = None
for excerpt in excerpts:
    if excerpt["id"] == evidence_id:
        cited_excerpt = excerpt

print("Theme:", suggestion["theme"])
print("Cited excerpt:", cited_excerpt)
print("Question:", suggestion["question_for_researcher"])

# ONE CHANGE: change e02 to
# "The food deliveries helped, but I avoided the group because meetings felt hostile."
```

i Text becomes a dictionary

`json.loads(raw_json)` parses the returned string into a Python dictionary. Square brackets retrieve the theme and cited ID. The `for` loop checks each source record until its ID matches; it then saves the actual excerpt. The output to assess is the pair `theme` plus `cited_excerpt`, not the theme alone.

13.1.6 Methodological check

Well-formed JSON makes the fields retrievable. The researcher must still decide whether the cited excerpt supports the theme and what contrary evidence changes it.

13.2 Workbook and environment links

[Open the Week 3 notebook in Google Colab](#) · [Download the notebook](#) · [Download the Python exercise as a .py file](#) · [Download the runnable offline bundle](#)

This page, the notebook and the .py download contain the same assessed routine. Use the notebook for the guided cell-by-cell walkthrough and the script when you want to run the complete routine from the repository root.

13.3 Take-home exercise

Estimated work and recording time: 30–45 minutes.

Use one chosen route, change only `e02` and rerun. Explain `ROUTE`, both `if/else` branches, `json.dumps`, the raw returned string, `json.loads` and the source lookup. Say whether the cited passage supports the proposed theme.

Submit one continuous screen recording with audio. Show the original run and the named change. Explain every operation's input, Python type and output; identify the access route; connect the methodological limit to one essential reading. Upload it using the [recording and submission instructions](#) by **5:00 p.m. Eastern on the Tuesday before the next class**. Never show or read the shared key. If a cached contingency was necessary, identify it.

14 Week 4 — What part of social experience becomes evidence?

14.1 This week's sociological question

What happens within a confrontation that allows violence to begin—or enables it to remain nonviolent—and what can a recording actually show?

14.2 Connection to the preceding material

Weeks 1–3 assumed that a text or excerpt already existed. This week moves back one stage. Camera position, transcription, synchronization, selection and professional ways of seeing all shape the record before a model receives it.

14.3 Learning outcomes

You should be able to separate observable conduct, interpretation and unobserved intention; explain Goodwin's account of professional vision; use Collins's situational theory to formulate a sequence question; reconstruct transitions across aligned sources; and identify what a multimodal model adds or obscures.

14.4 Essential readings and what each contributes

- **Methods:** Goodwin (1994), “Professional Vision,” *American Anthropologist* 96(3): 606–633. Read the complete article. It shows how coding, highlighting and material representations organize what members of a profession see. [DOI](#). ([Goodwin 1994](#))
- **Methods:** Nassauer and Legewie (2021), “Video Data Analysis,” *Sociological Methods & Research* 50(1): 135–174. Read the complete article. It gives a systematic account of selection, sequencing, transcription and inference when video becomes sociological data. [DOI](#). ([Nassauer and Legewie 2021](#))
- **Substantive application:** Collins (2013), “Entering and Leaving the Tunnel of Violence,” *Current Sociology* 61(2): 132–151. Read the complete article. Collins uses visual evidence and close sequence analysis to explain confrontation, tension and entrainment. [DOI](#). ([Collins 2013](#))

14.5 Additional readings grouped by purpose

- **For the technical grounding debate:** Bisk et al. (2020), “Experience Grounds Language,” *EMNLP. Paper*. ([Bisk et al. 2020](#))

14.6 Sociological and methodological background

Simmel’s sociology of the senses and Goffman’s interaction order make perception and situated conduct sociological topics. Goodwin goes further: relevant objects do not simply announce themselves. Practitioners learn to code a field of activity, highlight selected features and use material representations that make a judgment possible. His examples show that seeing is socially organized.

Collins’s theory of violent situations redirects attention from violent people to sequences of interaction. Confrontational tension and fear often inhibit violence; particular configurations allow participants to move through that barrier and sometimes enter a “tunnel” of emotional entrainment. This is a claim about timing and relation. A still image of contact cannot show the full sequence that preceded it.

Our empirical illustration is the Zidane–Materazzi confrontation in the 2006 World Cup final. Broadcast video supplies multiple views and a public chronology, but it does not give direct access to inner intention. The example allows us to distinguish what is visible—the turn, approach and headbutt—from Collins’s situational interpretation and from later biographical accounts.

14.7 Synthesis of the current LLM literature

Multimodal models can describe frames, transcribe audio and compare moments. They also arrive with pretrained recognition that may identify famous people or retrieve a culturally familiar narrative. That can be useful, but it can contaminate a task designed to inspect only visible conduct. World-model research asks whether models can learn predictive representations of physical environments; sociology asks an additional question: which social experiences, relations and professional categories have been made legible to the model at all?

The defensible workflow constrains the first pass to observable features, preserves time and source, and records later theoretical interpretation separately. We can then deliberately compare a frame-only call, a sequence call and a context-rich call.

14.8 Slides

The slides are available in the [online Week 4 chapter](#).

[Continue to the Week 4 Python work](#).

14.9 Summary and preparation for next week

Recorded data are selective, aligned and interpreted rather than transparent windows onto action. Week 5 changes the researcher's role: instead of observing a record, we generate candidate materials intended to alter what participants encounter.

15 Week 4 Python — Multimodal evidence

[Return to the Week 4 readings and slides.](#)

15.1 Python walkthrough: give a multimodal model visual evidence

Today's routine: Two interaction frames → one multimodal request → a structured visual description → evidence check.

Python introduced: file paths, ordered image lists, nested message content and structured visual descriptions.

Your change: replace the second frame and check which claims remain visibly supported.

The routine begins with the model input. It does not postpone the live call until after unrelated Python exercises. Predict what each block will create before running it.

15.1.1 Load the course settings and SDKs

Run the notebook's **Prepare the notebook environment** cell first. It installs missing SDKs in Colab, checks the local `uv` environment, adds the repository root to Python's import path and sets `IN_COLAB`. Adding the repository root is what makes the supplied `src.genai_soc.media` utility importable after Colab clones the repository.

This next cell reads the two centrally recorded model names and asks for the hosted key without displaying it. Treat the key entry as setup; never include it in the recording.

```
import json
import os
from getpass import getpass
from pathlib import Path

try:
    import ollama
    from openrouter import OpenRouter
except ModuleNotFoundError as error:
    raise ModuleNotFoundError(
        "A course SDK is missing. Open a terminal in the complete GenAI_Soc2026 "
        "folder, run 'uv sync --frozen', then run this task with 'uv run python'."
    ) from error

ROOT = Path.cwd()
```

```

while not (ROOT / "config" / "course_models.json").exists() and ROOT != ROOT.parent:
    ROOT = ROOT.parent

if not (ROOT / "config" / "course_models.json").exists():
    raise FileNotFoundError(
        "The complete GenAI_Soc2026 repository could not be found. A task or "
        "notebook downloaded by itself is not enough for local work. Open a terminal "
        "in the complete course folder and run this file from there."
    )

config = json.loads((ROOT / "config" / "course_models.json").read_text())
HOSTED_MODEL = config["hosted"]["model"]
LOCAL_MODEL = config["local"]["model"]

if not os.getenv("OPENROUTER_API_KEY"):
    os.environ["OPENROUTER_API_KEY"] = getpass("OpenRouter course key (hidden): ")

print("Hosted model:", HOSTED_MODEL)
print("Local model:", LOCAL_MODEL)

```

i Read this setup cell

Input: installed Python packages, `config/course_models.json`, and—if it is not already set—the hidden OpenRouter key. **Operations:** `import` makes an installed tool available; `Path.cwd()` gives Python the current folder; the `while` block walks upward until it finds the course configuration; `json.loads(...)` turns the file's JSON text into a dictionary; square brackets retrieve the two model names. **Output:** `HOSTED_MODEL` and `LOCAL_MODEL` are strings. `getpass(...)` accepts the key without echoing it. The folder-search code is supplied setup and is not assessed.

15.1.2 Load the supplied image transport utility and identify two frames

Run this block, print its output and explain what enters and leaves the operation. The next block uses the value created here.

```

from src.genai_soc.media import image_to_data_url

ROUTE = "openrouter" if IN_COLAB else "ollama" # Ollama is the local default
frame_1 = ROOT / "slides" / "session04" / "images" / "zidane_fine2_168.png"
frame_2 = ROOT / "slides" / "session04" / "images" / "zidane_fine2_169.png"
timestamps = ["00:00", "00:01"]
print(frame_1)
print(frame_2)
print("Both files exist:", frame_1.exists() and frame_2.exists())

```

i Files are inputs too

Each `Path` stores a location, not the image pixels themselves. `display(Image(...))` lets the researcher inspect the frames before asking a model about them. The supplied transport utility later converts a file to the data-URL representation OpenRouter accepts; students need to explain its file input and transport output, not binary encoding.

15.1.3 State the evidentiary boundary and the required JSON fields

Run this block, print its output and explain what enters and leaves the operation. The next block uses the value created here.

```
prompt = (
    "Compare these ordered frames. Describe only visible change. Do not name people, "
    "infer motive, or use remembered event knowledge. Return JSON with exactly "
    "observable_change, visible_evidence, and interpretation_withheld."
)
schema = {
    "type": "object",
    "properties": {
        "observable_change": {"type": "string"},
        "visible_evidence": {"type": "array", "items": {"type": "string"}},
        "interpretation_withheld": {"type": "string"},
    },
    "required": ["observable_change", "visible_evidence", "interpretation_withheld"],
    "additionalProperties": False,
}
```

i Constraint and schema

`instruction` tells the model to report visible change and withhold identity, motive and cause. `schema` is a nested dictionary describing the required fields and their types. It can constrain the form of a return; it cannot guarantee that a supposedly visible claim is actually present in a frame.

15.1.4 Send the images through the selected route

Run this block, print its output and explain what enters and leaves the operation. The next block uses the value created here.

```
if ROUTE == "openrouter":
    content = [
        {"type": "text", "text": prompt},
        {"type": "image_url", "image_url": {"url": image_to_data_url(frame_1)}},
        {"type": "image_url", "image_url": {"url": image_to_data_url(frame_2)}},
    ]
```



```

]
messages = [{"role": "user", "content": content}]
with OpenRouter(api_key=os.environ["OPENROUTER_API_KEY"]) as client:
    response = client.chat.send(
        model=HOSTED_MODEL, messages=messages, temperature=0,
        response_format={"type": "json_schema", "json_schema": {
            "name": "visible_sequence", "strict": True, "schema": schema,
        }},
    )
    raw_output = response.choices[0].message.content
else:
    messages = [{
        "role": "user", "content": prompt,
        "images": [str(frame_1), str(frame_2)],
    }]
    response = ollama.chat(think=False,
        model=LOCAL_MODEL, messages=messages, format=schema,
        options={"temperature": 0},
    )
    raw_output = response.message.content

print("Raw JSON text:", raw_output)

```

i Multimodal message and return

The message contains ordered text and image parts. The OpenRouter branch converts each path to a data URL; the Ollama branch supplies local path strings in its `images` field. Each branch requests the same schema and stores the returned JSON text in `raw_json`. Route-specific transport differs even though the research question is held fixed.

15.1.5 Parse the three fields and compare them with the images

Run this block, print its output and explain what enters and leaves the operation. The next block uses the value created here.

```

description = json.loads(raw_output)
print("Observable change:", description["observable_change"])
print("Visible evidence:", description["visible_evidence"])
print("Interpretation withheld:", description["interpretation_withheld"])

# ONE CHANGE: replace frame_2 with zidane_fine2_170.png and rerun.

```

i The substantive check

`json.loads(...)` creates a dictionary, then three key lookups retrieve the model's description, claimed visible evidence and withheld interpretation. Printing those values makes them available for frame-by-frame checking. A well-formed list of visible evidence is not proof that each item is visible.

15.1.6 Methodological check

The model can still recognize a famous event or import a motive despite the constraint. Each returned claim must be checked against pixels in the two displayed frames.

15.2 Workbook and environment links

[Open the Week 4 notebook in Google Colab](#) · [Download the notebook](#) · [Download the Python exercise as a .py file](#) · [Download the runnable offline bundle](#)

This page, the notebook and the .py download contain the same assessed routine. Use the notebook for the guided cell-by-cell walkthrough and the script when you want to run the complete routine from the repository root.

15.3 Take-home exercise

Estimated work and recording time: 30–45 minutes.

Use one route, replace the second frame with `zidane_fine2_170.png` and rerun. Explain each path, the image representation used by your route, the raw JSON and all three fields. Identify anything not visibly supported.

Submit one continuous screen recording with audio. Show the original run and the named change. Explain every operation's input, Python type and output; identify the access route; connect the methodological limit to one essential reading. Upload it using the [recording and submission instructions](#) by **5:00 p.m. Eastern on the Tuesday before the next class**. Never show or read the shared key. If a cached contingency was necessary, identify it.

Part II

Experiments and research encounters

16 Week 5 — Can generated media isolate a causal construct?

16.1 This week's sociological question

When an LLM drafts experimental material, how do we know that the candidates vary the intended construct rather than several things at once?

16.2 Connection to the preceding material

Week 4 examined how an unfolding situation becomes evidence. Week 5 examines the other side of the research encounter: how sociologists construct a situation, vignette, message or profile so that a comparison can support a causal claim.

16.3 Learning outcomes

You should be able to distinguish a vignette, audit study, confederate design and field experiment; explain factorial survey logic; identify semantic differences that act as treatment confounds; distinguish a generated candidate pool from a validated treatment set; and trace a candidate through explicit review rules.

16.4 Essential readings and what each contributes

- **Methods:** Wallander (2009), “25 Years of Factorial Surveys in Sociology,” *Social Science Research* 38(3): 505–520. Read the complete article. It establishes how sociologists combine experimentally varied vignette dimensions. DOI. (Wallander 2009)
- **Methods:** Dafoe, Zhang and Caughey (2018), “Information Equivalence in Survey Experiments,” *Political Analysis* 26(4): 399–416. Read the complete article. It gives us the language for asking whether alternative wordings convey the same non-focal information. DOI. (Dafoe, Zhang, and Caughey 2018)
- **Substantive application:** Bloemraad, Silva and Voss (2016), “Rights, Economics, or Family?” *Social Forces* 94(4): 1647–1674. Read the complete article. It tests competing frames in an immigration-policy experiment and supplies the substantive case used in the lecture. DOI. (Bloemraad, Silva, and Voss 2016)

16.5 Additional readings grouped by purpose

- **For generated policy messages tested with people:** Bai et al. (2025), “LLM-Generated Messages Can Persuade Humans on Policy Issues,” *Nature Communications*. DOI. ([Bai et al. 2025](#))
- **For adaptive candidate selection:** Velez (2025), “Crowdsourced Adaptive Surveys,” *Political Analysis*. DOI. ([Velez 2025](#))

16.6 Sociological and methodological background

Sociologists intervene in several ways. A vignette describes a situation and asks for a judgment; a factorial survey varies several attributes across vignettes; an audit study sends matched applications that differ on a focal signal; a confederate enacts a standardized role in a social setting. The concrete objects differ, but each design depends on controlling which feature changes and what comparison that change licenses.

Wallander’s review matters because generated text does not make this design logic obsolete. An LLM can produce a larger candidate pool, but every candidate still embeds word choice, tone, plausibility and social assumptions. If one paid-leave frame mentions “hard-working families” and another mentions “national productivity,” the conditions differ in more than a short label. Those semantic differences are potential confounds.

16.7 Synthesis of the current LLM literature

Recent work shows the value of generation without equating it with treatment readiness. Bai and colleagues generate and test policy messages on human participants. Velez separates an adaptive selection process from the population in which questions are evaluated. Dafoe and colleagues offer a precise standard for our review: non-focal information should be held equivalent across conditions. The LLM is useful upstream, where it can widen the pool. Human pretests and design checks remain necessary downstream.

16.8 Slides

The slides are available in the [online Week 5 chapter](#).

[Continue to the Week 5 Python work](#).

16.9 Summary and preparation for next week

Generation expands a design space. It does not decide which distinctions are causal, plausible or ethical. Week 6 makes the treatment path dynamic: the model chooses or composes the next question after seeing a participant’s answer.

17 Week 5 Python — Generating experimental materials

[Return to the Week 5 readings and slides.](#)

17.1 Python walkthrough: generate experimental materials

Today's routine: Fixed policy facts → two framing instructions → two generated treatments → confound check.

Python introduced: f-strings, named call arguments, integers, floats, temperature, output limits, latency and token metadata.

Your change: change only the frame while holding the facts and call settings fixed.

The routine begins with the model input. It does not postpone the live call until after unrelated Python exercises. Predict what each block will create before running it.

17.1.1 Load the course settings and SDKs

Run the notebook's **Prepare the notebook environment** cell first. It installs missing SDKs in Colab, checks the local uv environment, adds the repository root to Python's import path and sets `IN_COLAB`. Colab uses OpenRouter because it cannot reach the Ollama server on your computer.

This next cell reads the two centrally recorded model names and asks for the hosted key without displaying it. Treat the key entry as setup; never include it in the recording.

```
import json
import os
from getpass import getpass
from pathlib import Path

try:
    import ollama
    from openrouter import OpenRouter
except ModuleNotFoundError as error:
    raise ModuleNotFoundError(
        "A course SDK is missing. Open a terminal in the complete GenAI_Soc2026 "
        "folder, run 'uv sync --frozen', then run this task with 'uv run python'."
    ) from error
```

```

ROOT = Path.cwd()
while not (ROOT / "config" / "course_models.json").exists() and ROOT != ROOT.parent:
    ROOT = ROOT.parent

if not (ROOT / "config" / "course_models.json").exists():
    raise FileNotFoundError(
        "The complete GenAI_Soc2026 repository could not be found. A task or "
        "notebook downloaded by itself is not enough for local work. Open a terminal "
        "in the complete course folder and run this file from there."
    )

config = json.loads((ROOT / "config" / "course_models.json").read_text())
HOSTED_MODEL = config["hosted"]["model"]
LOCAL_MODEL = config["local"]["model"]

if not os.getenv("OPENROUTER_API_KEY"):
    os.environ["OPENROUTER_API_KEY"] = getpass("OpenRouter course key (hidden): ")

print("Hosted model:", HOSTED_MODEL)
print("Local model:", LOCAL_MODEL)

```

i Read this setup cell

Input: installed Python packages, `config/course_models.json`, and—if it is not already set—the hidden OpenRouter key. **Operations:** `import` makes an installed tool available; `Path.cwd()` gives Python the current folder; the `while` block walks upward until it finds the course configuration; `json.loads(...)` turns the file's JSON text into a dictionary; square brackets retrieve the two model names. **Output:** `HOSTED_MODEL` and `LOCAL_MODEL` are strings. `getpass(...)` accepts the key without echoing it. The folder-search code is supplied setup and is not assessed.

17.1.2 Choose a route and store the fixed facts and changing frame

Run this block, print its output and explain what enters and leaves the operation. The next block uses the value created here.

```

import time

ROUTE = "openrouter" if IN_COLAB else "ollama" # Ollama is the local default
shared_facts = (
    "The proposal would create a pathway to permanent legal status for undocumented "
    "immigrants who meet residency and background-check requirements."
)
frame = "rights"
temperature = 0.7
max_tokens = 140

```

i Separate what stays fixed

The facts and frame are separate strings so the exercise can change one while holding the other constant. `temperature` is a float and `maximum_output_tokens` is an integer. They are recorded as model-call inputs, not treated as substantive properties of the message.

17.1.3 Construct the prompt with an f-string

Run this block, print its output and explain what enters and leaves the operation. The next block uses the value created here.

```
prompt = (
    f"Write one 70-90 word survey message using a {frame} frame. "
    f"Use only these facts and return only the message:\n\n{shared_facts}"
)
messages = [{"role": "user", "content": prompt}]
print(prompt)
```

i How the f-string works

The leading `f` allows values inside braces to be inserted into a string. `{frame}` and `{facts}` are replaced by their current values. The output is one complete prompt; printing it is how we check that the intended frame changed and the factual material did not.

17.1.4 Make one timed call and preserve route-specific metadata

Run this block, print its output and explain what enters and leaves the operation. The next block uses the value created here.

```
started = time.perf_counter()
if ROUTE == "openrouter":
    with OpenRouter(api_key=os.environ["OPENROUTER_API_KEY"]) as client:
        response = client.chat.send(
            model=HOSTED_MODEL, messages=messages,
            temperature=temperature, max_tokens=max_tokens,
        )
        raw_output = response.choices[0].message.content
        input_tokens = getattr(response.usage, "prompt_tokens", None)
        output_tokens = getattr(response.usage, "completion_tokens", None)
else:
    response = ollama.chat(think=False,
        model=LOCAL_MODEL, messages=messages,
        options={"temperature": temperature, "num_predict": max_tokens},
    )
    raw_output = response.message.content
```



```

    input_tokens = response.prompt_eval_count
    output_tokens = response.eval_count
    latency_seconds = round(time.perf_counter() - started, 2)

print("Raw candidate:", raw_output.strip())
print("Input tokens:", input_tokens)
print("Output tokens:", output_tokens)
print("Latency seconds:", latency_seconds)

```

i Call inputs and observed outputs

`time.perf_counter()` records a clock value before and after the call; subtraction gives elapsed seconds. The selected branch supplies model, messages, temperature and output limit. It then retrieves the candidate text and, where available, token-use metadata. Latency and token counts describe this run, not the candidate's validity.

17.1.5 Save the design settings beside the candidate

Run this block, print its output and explain what enters and leaves the operation. The next block uses the value created here.

```

candidate_record = {
    "route": ROUTE,
    "frame": frame,
    "shared_facts": shared_facts,
    "temperature": temperature,
    "max_tokens": max_tokens,
    "raw_output": raw_output.strip(),
}
print(candidate_record)

# ONE CHANGE: change frame from "rights" to "economics" and nothing else.

```

i The candidate record

The dictionary stores the changing frame, fixed facts, route, model, settings, elapsed time and generated text together. This is the output needed for the later causal-design review. The named change asks for a second candidate; comparison should isolate intended framing from other semantic differences.

17.1.6 Methodological check

Holding the prompt variables fixed does not guarantee that the generated messages differ only in framing. Compare facts, certainty, tone, length and implied beneficiaries before treating them as experimental stimuli.

17.2 Workbook and environment links

[Open the Week 5 notebook in Google Colab](#) · [Download the notebook](#) · [Download the Python exercise as a .py file](#) · [Download the runnable offline bundle](#)

This page, the notebook and the .py download contain the same assessed routine. Use the notebook for the guided cell-by-cell walkthrough and the script when you want to run the complete routine from the repository root.

17.3 Take-home exercise

Estimated work and recording time: 30–45 minutes.

Run the rights and economics frames through one chosen route. Explain the f-string and every named call argument, then compare the candidates for semantic confounds and state what a human pretest must establish.

Submit one continuous screen recording with audio. Show the original run and the named change. Explain every operation's input, Python type and output; identify the access route; connect the methodological limit to one essential reading. Upload it using the [recording and submission instructions](#) by **5:00 p.m. Eastern on the Tuesday before the next class**. Never show or read the shared key. If a cached contingency was necessary, identify it.

18 Week 6 — What changes when a model starts asking the questions?

18.1 This week's sociological question

What kind of evidence does an interview produce, and what changes when a model can choose or write the next probe?

18.2 Connection to the preceding material

Week 5 generated fixed candidates before participants encountered them. In an adaptive interview, the participant's answer changes what comes next. The experimental treatment, data-collection instrument and interaction are now entangled.

18.3 Learning outcomes

You should be able to distinguish interview purposes and survey modes; explain specificity, range, depth and personal context in the focused interview; distinguish speech-to-text, a text interviewer and text-to-speech; compare an assigned protocol with the path a participant actually received; and audit the source and target of every adaptive probe.

18.4 Essential readings and what each contributes

- **Methods:** Merton and Kendall (1946), “The Focused Interview,” *American Journal of Sociology* 51(6): 541–557. Read the complete article. It supplies foundational criteria for evaluating what a probe elicits. [DOI](#). ([Merton and Kendall 1946](#))
- **Methods and LLMs:** Geiecke and Jaravel (2026), “The Rise of AI-Led Interviews in Social Science.” Read the complete paper. It presents a large text-and-voice implementation and explicit validation exercises. [Paper](#). ([Geiecke and Jaravel 2026](#))
- **Substantive application:** Rivera (2012), “Hiring as Cultural Matching,” *American Sociological Review* 77(6): 999–1022. Read the complete article. Rivera uses interviews and observation to show how evaluators construct cultural fit in elite hiring; it gives us a real case against which to assess adaptive probes. [DOI](#). ([Rivera 2012](#))

18.5 Additional readings grouped by purpose

- **For a qualitative chatbot case:** Jack, Cooper and Flower (2026), *Methodological Innovations*. DOI. (Jack, Cooper, and Flower 2026)
- **For comparison with adaptive survey selection:** Velez (2025), *Political Analysis*. DOI. (Velez 2025)

18.6 Sociological and methodological background

Interviews have never been neutral containers into which respondents deposit attitudes. Merton and Kendall’s focused interview sought detailed responses to a known situation while allowing unanticipated meanings to emerge. Later feminist and reflexive work made power, reciprocity and the interviewer’s social position part of the method. Standardized survey interviewing pursued comparability; CAPI moved administration onto an interviewer’s computer; CASI allowed self-administration; audio-CASI added recorded speech for accessibility and privacy. These modes change what respondents hear, disclose and understand.

Rivera’s study makes the evidentiary stakes concrete. Interviewers described cultural fit, while observation showed how evaluators converted shared tastes and experiences into judgments about candidates. The two sources do not simply duplicate one another. An adaptive interviewer might elicit a specific evaluative episode, but a leading probe could also supply the very explanation the study is meant to discover.

An LLM interviewer redistributes familiar decisions. A protocol can set topic boundaries and fixed questions. After an answer, a model can select a prepared follow-up or compose a new one. The second option can pursue specificity, but it can also lead, presuppose a cause, repeat itself or create incomparable paths across participants.

18.7 Synthesis of the current LLM literature

Geiecke and Jaravel use text and voice, so their system includes speech-to-text before the language model and text-to-speech after it. Jack and colleagues use a web chatbot built around Gemini Flash 1.5 and study both the interview data and researchers’ experience of the system. These implementations make privacy, retention and data ownership design questions: which service receives audio, where transcripts are stored, and whether data are reused. A local stack can reduce external transfer, but it demands hardware, model files, maintenance and usually slower inference.

The evaluation target also matters. Completion rate and conversational fluency do not establish qualitative richness. A controlled design can compare fixed questions, dynamic probing and member checking, but the concepts must be defined: a dynamic prober generates a follow-up from the answer; a member checker asks whether the interviewer’s understanding is accurate. In the worked example, the check is deliberately substantive: does the follow-up ask an evaluator to describe a concrete hiring decision, or does it prematurely name “cultural fit” as the cause?

18.8 Slides

The slides are available in the [online Week 6 chapter](#).

[Continue to the Week 6 Python work](#).

18.9 Summary and preparation for next week

An adaptive interviewer produces different question paths from the same protocol. After the midterm break, Week 7 moves from asking people questions to asking a model to answer as if it were a person.

19 Week 6 Python — Adaptive interview turns

[Return to the Week 6 readings and slides.](#)

19.1 Python walkthrough: build one adaptive interview turn

Today's routine: One participant answer → one adaptive probe → updated conversation → second adaptive probe.

Python introduced: conversation lists, roles, `.append(...)`, ordered state and repeated model calls.

Your change: replace the second participant answer and inspect the next probe.

The routine begins with the model input. It does not postpone the live call until after unrelated Python exercises. Predict what each block will create before running it.

19.1.1 Load the course settings and SDKs

Run the notebook's **Prepare the notebook environment** cell first. It installs missing SDKs in Colab, checks the local `uv` environment, adds the repository root to Python's import path and sets `IN_COLAB`. Colab uses OpenRouter because it cannot reach the Ollama server on your computer.

This next cell reads the two centrally recorded model names and asks for the hosted key without displaying it. Treat the key entry as setup; never include it in the recording.

```
import json
import os
from getpass import getpass
from pathlib import Path

try:
    import ollama
    from openrouter import OpenRouter
except ModuleNotFoundError as error:
    raise ModuleNotFoundError(
        "A course SDK is missing. Open a terminal in the complete GenAI_Soc2026 "
        "folder, run 'uv sync --frozen', then run this task with 'uv run python'."
    ) from error

ROOT = Path.cwd()
while not (ROOT / "config" / "course_models.json").exists() and ROOT != ROOT.parent:
```

```

ROOT = ROOT.parent

if not (ROOT / "config" / "course_models.json").exists():
    raise FileNotFoundError(
        "The complete GenAI_Soc2026 repository could not be found. A task or "
        "notebook downloaded by itself is not enough for local work. Open a terminal "
        "in the complete course folder and run this file from there."
    )

config = json.loads((ROOT / "config" / "course_models.json").read_text())
HOSTED_MODEL = config["hosted"]["model"]
LOCAL_MODEL = config["local"]["model"]

if not os.getenv("OPENROUTER_API_KEY"):
    os.environ["OPENROUTER_API_KEY"] = getpass("OpenRouter course key (hidden): ")

print("Hosted model:", HOSTED_MODEL)
print("Local model:", LOCAL_MODEL)

```

i Read this setup cell

Input: installed Python packages, `config/course_models.json`, and—if it is not already set—the hidden OpenRouter key. **Operations:** `import` makes an installed tool available; `Path.cwd()` gives Python the current folder; the `while` block walks upward until it finds the course configuration; `json.loads(...)` turns the file's JSON text into a dictionary; square brackets retrieve the two model names. **Output:** `HOSTED_MODEL` and `LOCAL_MODEL` are strings. `getpass(...)` accepts the key without echoing it. The folder-search code is supplied setup and is not assessed.

19.1.2 Choose a route and construct the opening interview history

Run this block, print its output and explain what enters and leaves the operation. The next block uses the value created here.

```

ROUTE = "openrouter" if IN_COLAB else "ollama" # Ollama is the local default
interview_messages = [
    {"role": "system", "content": (
        "Conduct a sociological interview. Ask one short follow-up about a concrete "
        "episode. Do not suggest a cause or put words in the participant's mouth."
    )},
    {"role": "user", "content": (
        "Participant: I spoke after the professor invited me to respond."
    )},
]
print(interview_messages)

```

i Conversation as ordered state

`history` is a list of dictionaries in chronological order. Roles say whether an item is an interviewing instruction or participant speech. This complete list becomes the input to the first call. Reordering or omitting an item would change what the model can use.

19.1.3 Make the first call through the selected route

Run this block, print its output and explain what enters and leaves the operation. The next block uses the value created here.

```
if ROUTE == "openrouter":
    with OpenRouter(api_key=os.environ["OPENROUTER_API_KEY"]) as client:
        first_response = client.chat.send(
            model=HOSTED_MODEL, messages=interview_messages, temperature=0,
        )
        first_raw = first_response.choices[0].message.content
else:
    first_response = ollama.chat(think=False,
        model=LOCAL_MODEL, messages=interview_messages,
        options={"temperature": 0},
    )
    first_raw = first_response.message.content
print("First raw return:", first_raw)
first_probe = first_raw.strip()
print("First probe:", first_probe)
```

i The first adaptive probe

The if/else branch sends the same history through the chosen route. Both branches store returned text in `probe_one`. That string is the model's proposed next interviewer turn; the later check asks whether it follows the participant's answer without supplying a cause.

19.1.4 Append the realized probe and a second participant answer

Run this block, print its output and explain what enters and leaves the operation. The next block uses the value created here.

```
interview_messages.append({"role": "assistant", "content": first_probe})
second_answer = "Participant: I felt safer because she made room for me to speak."
interview_messages.append({"role": "user", "content": second_answer})
print("Messages before second call:", len(interview_messages))
print(interview_messages)
```


i What append changes

`.append(...)` mutates the existing list by adding one dictionary at its end. The first append records what the model asked; the second records the participant's next answer. Printing the history shows the new ordered input that the second call will receive.

19.1.5 Make the second call with the expanded history

Run this block, print its output and explain what enters and leaves the operation. The next block uses the value created here.

```
if ROUTE == "openrouter":
    with OpenRouter(api_key=os.environ["OPENROUTER_API_KEY"]) as client:
        second_response = client.chat.send(
            model=HOSTED_MODEL, messages=interview_messages, temperature=0,
        )
        second_raw = second_response.choices[0].message.content
else:
    second_response = ollama.chat(think=False,
        model=LOCAL_MODEL, messages=interview_messages,
        options={"temperature": 0},
    )
    second_raw = second_response.message.content
print("Second raw return:", second_raw)
second_probe = second_raw.strip()
print("Second probe:", second_probe)

# ONE CHANGE: replace second_answer with
# "Participant: I spoke when there was a pause." and rerun from append onward.
```

i Why the second call differs

The call syntax is unchanged, but `history` now contains two additional turns. The returned `probe_two` can therefore respond to the new answer. Comparing the two probes checks continuity, leading language and repetition; it does not by itself establish interview quality.

19.1.6 Methodological check

Inspect whether each probe follows the participant's words, asks for a concrete episode, avoids repetition and does not name a cause in advance.

19.2 Workbook and environment links

[Open the Week 6 notebook in Google Colab](#) · [Download the notebook](#) · [Download the Python exercise as a .py file](#) · [Download the runnable offline bundle](#)

This page, the notebook and the .py download contain the same assessed routine. Use the notebook for the guided cell-by-cell walkthrough and the script when you want to run the complete routine from the repository root.

19.3 Take-home exercise

Estimated work and recording time: 30–45 minutes.

Use one route, make both calls, then change only the second participant answer. Explain the history before each call, both `.append()` operations and why the second call has more context than the first.

Submit one continuous screen recording with audio. Show the original run and the named change. Explain every operation's input, Python type and output; identify the access route; connect the methodological limit to one essential reading. Upload it using the [recording and submission instructions](#) by **5:00 p.m. Eastern on the Tuesday before the next class**. Never show or read the shared key. If a cached contingency was necessary, identify it.

Part III

Synthetic populations and social simulation

20 Week 7 — Whose attitudes do synthetic respondents represent?

20.1 This week's sociological question

What kind of evidence is produced when a model answers a survey question as if it were a person?

20.2 Connection to the preceding material

Week 6 kept human respondents but changed the interviewer. Week 7 changes the source of the answer. That makes the relationship between a record and a target population the central problem.

20.3 Learning outcomes

You should be able to distinguish a human respondent, aggregate prediction, demographic persona and interview-grounded agent; explain why probability sampling matters; compare means, distributions and within-group variation; trace one structured OpenRouter request; and state what a schema validates.

20.4 Essential readings and what each contributes

- **Methods:** Kozlowski and Evans (2025), “Simulating Subjects,” *Sociological Methods & Research*. Read the complete article. It frames the promise and peril of using LLMs as stand-ins for social agents. DOI. ([Kozlowski and Evans 2025](#))
- **Methods and validation:** Boelaert et al. (2025), “Machine Bias,” *Sociological Methods & Research*. Read the complete article. It tests how language models answer opinion polls and compares their response patterns with people. DOI. ([Boelaert et al. 2025](#))
- **Substantive application:** Igarashi, Kano and Miwa (2025), “ChatGPT versus Humans in Judging Discriminatory Scenarios,” *Humanities and Social Sciences Communications* 12: 1776. Read the complete article. Humans and GPT-4 judge the same experimentally varied scenarios of unequal treatment, giving us a substantive discrimination case and a direct comparison between human and model response patterns. DOI. ([Igarashi, Kano, and Miwa 2025](#))

20.5 Additional readings grouped by purpose

- **For identity flattening:** Wang, Morgenstern and Dickerson (2025), *Nature Machine Intelligence* 7: 400–411. DOI. (Wang, Morgenstern, and Dickerson 2025)
- **For survey augmentation rather than replacement:** Kim and Lee (2026), “AI-Augmented Surveys.” Preprint. (Kim and Lee 2026)

20.6 Sociological and methodological background

Survey inference depends on more than asking a question. Probability sampling links observed respondents to a target population; questionnaire design shapes the answers; total survey error directs attention to coverage, sampling, nonresponse, measurement and processing. Simulmatics provides an instructive prehistory: in 1960 the company used survey segments and computational models to advise the Kennedy campaign. The ambition to model an electorate is older than LLMs, as are the problems created by missing and poorly measured groups.

Public opinion theory adds another caution. Zaller treats a survey response as something assembled at the time from accessible considerations, not a stable value stored unchanged in a person. A demographic persona such as “a 42-year-old Black woman in Georgia” therefore does not reproduce the experiences, networks and information environment through which a human answer is formed.

Igarashi, Kano and Miwa turn this general warning into an empirical comparison. Human respondents and GPT-4 judge the same experimentally varied scenarios of unequal treatment. This lets us ask two separate questions: whether the model gives similar average judgments, and whether it responds to changes in the scenario in the same way as people do. Similarity on one does not guarantee similarity on the other.

20.7 Synthesis of the current LLM literature

The literature now contains several distinct designs. A model can predict an aggregate distribution, answer once per persona, predict a held-out answer for a real respondent, or act through an interview-grounded record. These are not interchangeable. Matching the modal response does not recover the distribution; matching the distribution does not recover individual pairings; aggregate resemblance can coexist with flattened subgroup heterogeneity. Igarashi and colleagues add another target: responsiveness to an experimental treatment. Validation must name which of these targets matters before calculating a score.

20.8 Slides

The slides are available in the [online Week 7 chapter](#).

[Continue to the Week 7 Python work](#).

20.9 Summary and preparation for next week

A synthetic response is conditioned model output, not a sampled person. Week 8 asks a narrower but still ambitious question: can observed human information help predict answers or results that were not observed?

21 Week 7 Python — Structured synthetic responses

[Return to the Week 7 readings and slides.](#)

21.1 Python walkthrough: request a structured synthetic response

Today's routine: One deidentified profile → one structured prediction → three schema checks → held-out comparison.

Python introduced: JSON Schema, required fields, numeric lists, `len(...)`, `sum(...)` and Boolean checks.

Your change: change only `age_group`, rerun and compare the probability lists.

The routine begins with the model input. It does not postpone the live call until after unrelated Python exercises. Predict what each block will create before running it.

21.1.1 Load the course settings and SDKs

Run the notebook's **Prepare the notebook environment** cell first. It installs missing SDKs in Colab, checks the local `uv` environment, adds the repository root to Python's import path and sets `IN_COLAB`. Colab uses OpenRouter because it cannot reach the Ollama server on your computer.

This next cell reads the two centrally recorded model names and asks for the hosted key without displaying it. Treat the key entry as setup; never include it in the recording.

```
import json
import os
from getpass import getpass
from pathlib import Path

try:
    import ollama
    from openrouter import OpenRouter
except ModuleNotFoundError as error:
    raise ModuleNotFoundError(
        "A course SDK is missing. Open a terminal in the complete GenAI_Soc2026 "
        "folder, run 'uv sync --frozen', then run this task with 'uv run python'."
    ) from error
```

```

ROOT = Path.cwd()
while not (ROOT / "config" / "course_models.json").exists() and ROOT != ROOT.parent:
    ROOT = ROOT.parent

if not (ROOT / "config" / "course_models.json").exists():
    raise FileNotFoundError(
        "The complete GenAI_Soc2026 repository could not be found. A task or "
        "notebook downloaded by itself is not enough for local work. Open a terminal "
        "in the complete course folder and run this file from there."
    )

config = json.loads((ROOT / "config" / "course_models.json").read_text())
HOSTED_MODEL = config["hosted"]["model"]
LOCAL_MODEL = config["local"]["model"]

if not os.getenv("OPENROUTER_API_KEY"):
    os.environ["OPENROUTER_API_KEY"] = getpass("OpenRouter course key (hidden): ")

print("Hosted model:", HOSTED_MODEL)
print("Local model:", LOCAL_MODEL)

```

i Read this setup cell

Input: installed Python packages, `config/course_models.json`, and—if it is not already set—the hidden OpenRouter key. **Operations:** `import` makes an installed tool available; `Path.cwd()` gives Python the current folder; the `while` block walks upward until it finds the course configuration; `json.loads(...)` turns the file's JSON text into a dictionary; square brackets retrieve the two model names. **Output:** `HOSTED_MODEL` and `LOCAL_MODEL` are strings. `getpass(...)` accepts the key without echoing it. The folder-search code is supplied setup and is not assessed.

21.1.2 Choose a route and store one deidentified profile and survey item

Run this block, print its output and explain what enters and leaves the operation. The next block uses the value created here.

```

ROUTE = "openrouter" if IN_COLAB else "ollama" # Ollama is the local default
profile = {
    "age_group": "30-44",
    "education": "bachelor's degree",
    "region": "South",
    "party_identification": "independent",
}
question = (
    "On a scale from 1 (very liberal) to 7 (very conservative), "
    "where would you place yourself?"

```



```
)
held_out_human_answer = 4
print(profile)
print(question)
```

i Profile, question and held-out answer

`profile` is a dictionary containing public-use teaching attributes; `survey_item` is the exact question string. `held_out_human_answer` is stored separately and is not included in the prompt. This separation prevents the model from seeing the answer it is meant to predict.

21.1.3 Define the exact output structure

Run this block, print its output and explain what enters and leaves the operation. The next block uses the value created here.

```
schema = {
    "type": "object",
    "properties": {
        "predicted_category": {"type": "integer", "minimum": 1, "maximum": 7},
        "probabilities": {
            "type": "array", "items": {"type": "number"},
            "minItems": 7, "maxItems": 7,
        },
    },
    "required": ["predicted_category", "probabilities"],
    "additionalProperties": False,
}
prompt = (
    "Predict this deidentified respondent's answer. Return seven probabilities in "
    "order from 1 to 7. Profile: " + json.dumps(profile) + " Item: " + question
)
messages = [{"role": "user", "content": prompt}]
```

i Reading the JSON Schema

The schema requires an integer category from 1 to 7 and a seven-number probability list. `required` says both fields must appear; `additionalProperties: False` prohibits extra fields. These rules concern readable structure and numeric bounds, not whether the prediction represents people well.

21.1.4 Make the selected structured-output call

Run this block, print its output and explain what enters and leaves the operation. The next block uses the value created here.

```

if ROUTE == "openrouter":
    with OpenRouter(api_key=os.environ["OPENROUTER_API_KEY"]) as client:
        response = client.chat.send(
            model=HOSTED_MODEL, messages=messages, temperature=0,
            response_format={"type": "json_schema", "json_schema": {
                "name": "polviews_prediction", "strict": True, "schema": schema,
            }},
        )
        raw_output = response.choices[0].message.content
else:
    response = ollama.chat(think=False,
        model=LOCAL_MODEL, messages=messages, format=schema,
        options={"temperature": 0},
    )
    raw_output = response.message.content
print("Raw JSON text:", raw_output)

```

i Two ways to request the same structure

The prompt joins the profile and exact item into one message. OpenRouter places the schema in `response_format`; Ollama receives it through `format`. Both branches preserve the returned JSON text in `raw_json` before any field is extracted.

21.1.5 Parse, check structure and only then compare with the held-out answer

Run this block, print its output and explain what enters and leaves the operation. The next block uses the value created here.

```

prediction = json.loads(raw_output)
probabilities = prediction["probabilities"]
seven_values = len(probabilities) == 7
sum_is_close = abs(sum(probabilities) - 1.0) < 0.02
matches_human = prediction["predicted_category"] == held_out_human_answer
print("Predicted category:", prediction["predicted_category"])
print("Seven probabilities:", seven_values)
print("Sum close to one:", sum_is_close)
print("Held-out human answer:", held_out_human_answer)
print("Exact match:", matches_human)

# ONE CHANGE: change age_group to "18-29" and rerun.

```

i Structure check versus evidence check

Parsing creates a dictionary. `len(probabilities)` counts entries and `sum(probabilities)` adds them; comparisons produce Booleans checking seven values, an approximate total of one

and values between zero and one. Only after those mechanical checks is `predicted_category` compared with the held-out human answer. One match is not population validation.

21.1.6 Methodological check

A valid schema means Python can retrieve the required fields. One exact match does not establish individual accuracy, group fidelity or population representation.

21.2 Workbook and environment links

[Open the Week 7 notebook in Google Colab](#) · [Download the notebook](#) · [Download the Python exercise as a .py file](#) · [Download the runnable offline bundle](#)

This page, the notebook and the `.py` download contain the same assessed routine. Use the notebook for the guided cell-by-cell walkthrough and the script when you want to run the complete routine from the repository root.

21.3 Take-home exercise

Estimated work and recording time: 30–45 minutes.

Use one route, run the original profile, change only `age_group` and rerun. Explain every schema field, the raw JSON, the parsed list and all three checks. State why neither prediction establishes representativeness.

Submit one continuous screen recording with audio. Show the original run and the named change. Explain every operation's input, Python type and output; identify the access route; connect the methodological limit to one essential reading. Upload it using the [recording and submission instructions](#) by **5:00 p.m. Eastern on the Tuesday before the next class**. Never show or read the shared key. If a cached contingency was necessary, identify it.

22 Week 8 — Can LLMs recover opinions we did not observe?

22.1 This week's sociological question

When a model predicts an unobserved answer or study result, what exactly has been recovered?

22.2 Connection to the preceding material

Week 7 asked what a synthetic response represents. Week 8 narrows the task: begin with real human data, hide some of it and ask a model to predict what was not observed. This makes validation possible, but only if we specify whether the target is a person's answer, a population distribution, a group difference or a relationship among variables.

22.3 Learning outcomes

You should be able to distinguish imputation, retrodiction and prediction of never-asked opinions; separate individual accuracy from aggregate resemblance; explain why a plausible record can still distort a population; trace the first `for` loop one case at a time; and compare hosted and local model results without treating either route as the truth.

22.4 Essential readings and what each contributes

- **Methods:** Kim and Lee (2026), “AI-Augmented Surveys.” Read the complete paper. It uses observed survey answers to predict unasked items for real respondents and distinguishes augmentation from wholesale replacement. [Preprint](#). ([Kim and Lee 2026](#))
- **Methods and validation:** Xie et al. (2026), “Evaluating the Statistical Realism of LLM-Generated Social Science Data.” Read the complete article. It asks whether success on a selected statistic survives broader tests of joint distributions and relationships among variables. [DOI](#). ([Xie et al. 2026](#))
- **Substantive application:** Lax and Phillips (2009), “Gay Rights in the States,” *American Political Science Review* 103(3): 367–386. Read the complete article. It applies multilevel regression and poststratification to state opinion and asks whether policy responds to public preferences on gay rights. This is an established comparison for claims about recovering quantities we did not directly observe. [DOI](#). ([Lax and Phillips 2009](#))

22.5 Additional readings grouped by purpose

- **For forecasting experimental results:** Ashokkumar et al. (2026), “Testing Theories of Human Behaviour at Scale with an AI Research Agent.” DOI. ([Ashokkumar et al. 2026](#))
- **For within-group heterogeneity:** Wang, Morgenstern and Dickerson (2025), “Large Language Models That Replace Human Participants Can Harmfully Misportray and Flatten Identity Groups.” DOI. ([Wang, Morgenstern, and Dickerson 2025](#))

22.6 Sociological and methodological background

Social scientists already estimate quantities they do not observe directly. Lax and Phillips combine actual survey responses, a multilevel model and census population counts to estimate state opinion, then use those estimates to study policy responsiveness. Their evidence chain matters: observed answers and known population margins constrain the estimate, validation is explicit, and the estimated opinion serves a substantive analysis rather than becoming an end in itself.

An LLM prediction changes this chain. Kim and Lee train on questions, respondents’ other answers and survey years to study three tasks: imputing a missing answer, retrodicting an item not asked in a particular wave and predicting an opinion never asked in the survey. These tasks expose the model to different amounts of relevant evidence and therefore warrant different claims.

22.7 Synthesis of the current LLM literature

Xie and colleagues evaluate whether generated datasets preserve distributions, bivariate associations, multivariate relationships, life-event sequences and links between sequences and covariates. Their results show why “the synthetic data look realistic” is too vague. A model can approximate a marginal distribution while compressing subgroup variation or changing relationships among variables. Kim and Lee likewise separate individual- and aggregate-level performance. Ashokkumar and colleagues extend prediction to experimental findings, but that is a different target again.

The coding example keeps the distinction visible. Five real teaching profiles are passed through OpenRouter and Ollama. The human answer is withheld from each prompt, every raw return is saved, and individual exact matches are calculated separately from the aggregate mean. Five records are not a validation study; they are enough to learn how a repeated model routine works and why the metric must match the claim.

22.8 Slides

The slides are available in the [online Week 8 chapter](#).

[Continue to the Week 8 Python work](#).

22.9 Summary and preparation for next week

A model can be evaluated against answers it did not see, but the result is always relative to a stated target. Week 9 moves from repeated predictions to agents that update an action after observing another actor—and introduces functions only after the inputs and outputs of the model routine are familiar.

23 Week 8 Python — Repeat predictions and compare routes

[Return to the Week 8 readings and slides.](#)

23.1 Python walkthrough: repeat one prediction across five cases

Today's routine: Five deidentified profiles → the same request through OpenRouter and Ollama → one saved record per case and route → individual and aggregate comparisons.

Python introduced: the first `for` loop around a model call, empty lists, loop variables, `.append(...)`, counters and division.

Your change: change only the education field for case p02, rerun and compare the two kinds of result.

Week 8 requires both routes again. Run it on a computer that has Ollama and the course model installed. Colab can run the OpenRouter branch, but it cannot reach Ollama running on your laptop; if your machine cannot run the local model, use the paired or instructor machine in class.

23.1.1 Load the settings and start with five records

Run the notebook setup cell first. It installs missing packages in Colab, locates the repository locally and reads the centrally recorded model names. The setup code is supplied and is not the assessed Python for this week.

```
profiles = [
    {"id": "p01", "age_group": "18-29", "education": "some college",
     "region": "Northeast", "party": "Democrat", "human_answer": 2},
    {"id": "p02", "age_group": "30-44", "education": "bachelor's",
     "region": "South", "party": "Independent", "human_answer": 4},
    # Three further records are supplied in the notebook.
]

question = "Place yourself from 1 (very liberal) to 7 (very conservative)."
print("Number of profiles:", len(profiles))
```

i Read the input before running it

`profiles` is a list. Each item is a dictionary representing one deidentified case. `human_answer` is retained for the later comparison but must be removed from the dictionary sent to the model. `question` is a string. `len(profiles)` returns the integer number of cases.

23.1.2 Create two empty result lists

```
hosted_results = []
local_results = []

print(hosted_results)
print(local_results)
```

i Input and output

There is no data input yet. Each pair of square brackets creates an empty list. After each model call, `.append(...)` will add one result dictionary. Keeping two lists means we do not lose which route produced a prediction.

23.1.3 Follow one trip through the loop

```
for profile in profiles:
    public_profile = {
        "age_group": profile["age_group"],
        "education": profile["education"],
        "region": profile["region"],
        "party": profile["party"],
    }

    prompt = (
        "Predict the survey answer. Profile: "
        + json.dumps(public_profile)
        + " Item: "
        + question
    )
    messages = [{"role": "user", "content": prompt}]
```

i What the loop changes

On the first pass, `profile` means `profiles[0]`; on the second, it means `profiles[1]`. `public_profile` copies only the fields the model may see, so `human_answer` is absent.

`json.dumps(...)` turns that dictionary into text. The output of this block is one `messages` list ready for a model call.

23.1.4 Make both calls and keep the raw text

```
with OpenRouter(api_key=os.environ["OPENROUTER_API_KEY"]) as client:
    hosted_response = client.chat.send(
        model=HOSTED_MODEL,
        messages=messages,
        temperature=0,
        response_format={
            "type": "json_schema",
            "json_schema": {
                "name": "opinion_prediction",
                "strict": True,
                "schema": schema,
            },
        },
    )
    hosted_raw = hosted_response.choices[0].message.content

    local_response = ollama.chat(think=False,
        model=LOCAL_MODEL,
        messages=messages,
        format=schema,
        options={"temperature": 0},
    )
    local_raw = local_response.message.content

    print(profile["id"], "hosted:", hosted_raw, "local:", local_raw)
```

i Two routes, the same research input

Both calls receive the same `messages` list and a schema requiring an answer from 1 to 7. OpenRouter sends the request to the hosted model; Ollama sends it to the local model already running on the computer. `hosted_raw` and `local_raw` are strings. Printing them before parsing makes the original returns visible.

23.1.5 Parse and append one record to each list

```
hosted_prediction = json.loads(hosted_raw)["predicted_category"]
local_prediction = json.loads(local_raw)["predicted_category"]
```

```

hosted_results.append({
    "id": profile["id"],
    "prediction": hosted_prediction,
    "human": profile["human_answer"],
})
local_results.append({
    "id": profile["id"],
    "prediction": local_prediction,
    "human": profile["human_answer"],
})

```

i What `.append(...)` does

`json.loads(...)` turns each JSON string into a dictionary; square brackets retrieve its predicted category. Each `.append(...)` adds one new dictionary to the end of a result list. After five passes through the loop, each list contains five records.

23.1.6 Count exact matches, then compare the aggregate means

```

hosted_matches = 0
for record in hosted_results:
    if record["prediction"] == record["human"]:
        hosted_matches = hosted_matches + 1

human_total = 0
hosted_total = 0
for record in hosted_results:
    human_total = human_total + record["human"]
    hosted_total = hosted_total + record["prediction"]

human_mean = human_total / len(hosted_results)
hosted_mean = hosted_total / len(hosted_results)

print("Hosted exact matches:", hosted_matches)
print("Human mean:", human_mean)
print("Hosted mean:", hosted_mean)

```

i These are different checks

The first loop asks whether the prediction and human answer match case by case. The second adds the values before dividing by the number of records. A close mean can coexist with poor individual prediction because over-predictions and under-predictions can cancel. The notebook repeats both calculations for the local route.

23.1.7 Methodological check

Five teaching cases cannot establish individual accuracy, subgroup fidelity or population validity. The exercise demonstrates how to preserve matched records and why a claim about an aggregate requires a different check from a claim about people.

23.2 Workbook and environment links

[Open the Week 8 notebook in Google Colab](#) · [Download the notebook](#) · [Download the Python exercise as a .py file](#) · [Download the runnable offline bundle](#)

This page, the notebook and the .py download contain the same assessed routine. Use the notebook for the guided cell-by-cell walkthrough and the script when running the complete routine from the repository root.

23.3 Take-home exercise

Estimated work and recording time: 35–50 minutes.

Run all five profiles through OpenRouter and Ollama. Change only p02's education from "bachelor's" to "high school", rerun, and trace that case through one full iteration for each route. Explain the input and output of the loop, both raw returns, both appended records, the exact-match counts and the aggregate means. State why agreement on the mean would not establish correct individual prediction or population representation.

Submit one continuous screen recording with audio using the [recording and submission instructions](#) by **5:00 p.m. Eastern on the Tuesday before the next class**. Never show or read the shared key. If you used the paired or instructor machine for Ollama, say so; hardware limitations do not reduce the completion mark.

24 Week 9 — How can micro-level interactions generate macro-level order?

24.1 Question

What must we specify before an apparent group pattern can be explained as the result of interacting agents?

24.2 Why this week belongs here

The course moves from model-generated responses to actors that observe one another and update over time. The sociological problem is micro-to-macro explanation, not simply whether agents can produce dialogue.

24.3 Provisional core readings

- Barrie et al. (2026), §§10.4.1–10.4.3. [Download chapter PDF](#). ([Barrie et al. 2026](#))
- Macy and Willer (2002), “From Factors to Actors.” ([Macy and Willer 2002](#))
- Park et al. (2023), “Generative Agents.” ([Park et al. 2023](#))
- Further comparison: Chuang et al. (2024) and Ashery, Aiello and Baronchelli (2025). ([Chuang et al. 2024](#); [Ashery, Aiello, and Baronchelli 2025](#))

24.4 Planned chapter and class arc

Reconstruct the explanatory logic of classical ABMs; define state, observation, policy, network and timing; examine what memory, reflection and language policies add; compare generative-agent claims with transparent rule-based baselines.

[See the provisional Week 9 Python plan.](#)

24.5 Development status

This is a planning page. Slides, the full literature synthesis, workbook and take-home exercise will be built and reviewed later.

25 Week 9 Python — Putting a model call inside a function

[Return to the Week 9 readings outline.](#)

This page records the coding direction only. The full walkthrough, notebook and completion task will be added after the week's literature, slides and examples have been developed together.

25.1 Planned code trace

Research task: Ask one simulated actor to choose its next action after observing another actor.

Python sequence: `def`, explicit parameters, local variables, `return` and calling one function with changed arguments. The runnable notebook follows input → messages → route → call → raw return → parsed output → check.

25.2 Planned completion recording

Explain what enters each of the four function parameters, what remains local inside the function and what the returned dictionary contains. Compare the two calls without treating the model's reason as observed motivation. Submit one narrated screen recording using the [recording and submission instructions](#) by 5:00 p.m. Eastern on the Tuesday before the next class. The key must not appear.

25.3 Development status

The page is deliberately provisional. It does not yet publish an assessed notebook or task.

26 Week 10 — When can agent diversity and debate produce collective intelligence?

26.1 Question

If a group of agents converges on a good answer, what evidence shows that interaction produced the result?

26.2 Why this week belongs here

Week 9 specifies interaction. Week 10 asks how to attribute a collective result when the same endpoint might come from shared model priors, leakage, prompting or selection of a favorable run.

26.3 Provisional core readings

- Barrie et al. (2026), §§10.4.4–10.4.5. [Download chapter PDF](#). ([Barrie et al. 2026](#))
- Ashery, Aiello and Baronchelli (2025), “Emergent Social Conventions and Collective Bias in LLM Populations.” ([Ashery, Aiello, and Baronchelli 2025](#))
- Barrie and Törnberg (2025), “Emergent LLM Behaviors Are Observationally Equivalent to Data Leakage.” ([Barrie and Törnberg 2025](#))
- Further comparison: Kim et al. (2026) and Chuang et al. (2024). ([Kim, Lai, et al. 2026](#); [Chuang et al. 2024](#))

26.4 Planned chapter and class arc

Distinguish convergence from correctness; compare repeated runs and no-interaction baselines; introduce observational equivalence; evaluate contamination probes and alternative explanations for collective performance.

[See the provisional Week 10 Python plan.](#)

26.5 Development status

This is a planning page. The two-session simulation arc will be finalized before slides and student tasks are published.

27 Week 10 Python — Running a small interaction repeatedly

[Return to the Week 10 readings outline.](#)

This page records the coding direction only. The full walkthrough, notebook and completion task will be added after the week's literature, slides and examples have been developed together.

27.1 Planned code trace

Research task: Compare a no-interaction baseline with a condition in which each simulated actor sees the preceding action.

Python sequence: repeated function calls, `range(...)`, accumulated run lists, conditions and small outcome summaries. The runnable notebook follows input → messages → route → call → raw return → parsed output → check.

27.2 Planned completion recording

Explain one actor update, one full group run and how `range(3)` creates repeated evidence. Reverse actor order and discuss what the changed or unchanged outcomes establish. Submit one narrated screen recording using the [recording and submission instructions](#) by **5:00 p.m. Eastern on the Tuesday before the next class**. The key must not appear.

27.3 Development status

The page is deliberately provisional. It does not yet publish an assessed notebook or task.

Part IV

Research agents, replication and model audits

28 Week 11 — Who—or what—produces research data in an agentic workflow?

28.1 Question

When a research agent searches, extracts, analyzes and writes, how can a human verify where each claim came from?

28.2 Why this week belongs here

The first research-agent session introduces planning, tool use, state and long-horizon failure. It connects the division of research labor to claim-level provenance.

28.3 Provisional core readings

- Barrie et al. (2026), §10.5. [Download chapter PDF](#). ([Barrie et al. 2026](#))
- Lu et al. (2026), “Towards End-to-End Automation of AI Research.” ([Lu et al. 2026](#))
- *Nature Machine Intelligence* (2026), “Multi-Agent AI Systems Need Transparency.” ([Nature Machine Intelligence 2026](#))
- Further reporting context: Feuerriegel et al. (2026). ([Feuerriegel et al. 2026](#))

28.4 Planned chapter and class arc

Introduce agent trajectories and long-horizon tasks; examine current systems and benchmarks, including CRUX and recent Meta work; localize errors across retrieval, extraction, code and synthesis; finish with a human-verifiable claim/source table.

[See the provisional Week 11 Python plan.](#)

28.5 Development status

This is a planning page. The current thin generated deck is deliberately not published; the chapter, slides and workbook will follow a fresh literature review.

29 Week 11 Python — A transparent research-agent tool call

[Return to the Week 11 readings outline.](#)

This page records the coding direction only. The full walkthrough, notebook and completion task will be added after the week's literature, slides and examples have been developed together.

29.1 Planned code trace

Research task: Let a model request one search of a supplied course corpus, then preserve the tool request, source return and drafted claim.

Python sequence: functions as tools, arguments, returned dictionaries, conditional dispatch and provenance records. The runnable notebook follows input → messages → route → call → raw return → parsed output → check.

29.2 Planned completion recording

Change the question and trace question → raw tool request → function argument → returned source record → raw claim → trajectory. Reject an unresolved or unsupported claim. Submit one narrated screen recording using the [recording and submission instructions](#) by **5:00 p.m. Eastern on the Tuesday before the next class**. The key must not appear.

29.3 Development status

The page is deliberately provisional. It does not yet publish an assessed notebook or task.

30 Week 12 — What can we reproduce, and what remains opaque?

30.1 Question

If an LLM-assisted result changes, which parts of the discrepancy can we explain from the saved record?

30.2 Why this week belongs here

This second research-agent session turns to replication, provider dependence and the practical record needed to understand a rerun. It remains low-stakes because of the holiday week.

30.3 Provisional core readings

- Barrie et al. (2026), §10.7. [Download chapter PDF](#). ([Barrie et al. 2026](#))
- Feuerriegel et al. (2026), reporting checklist. ([Feuerriegel et al. 2026](#))
- Barrie, Palmer and Spirling (2025), “Replication for Language Models.” ([Barrie, Palmer, and Spirling 2025](#))
- Further multi-agent context: *Nature Machine Intelligence* (2026). ([Nature Machine Intelligence 2026](#))

30.4 Planned chapter and class arc

Distinguish repeatability, robustness and replication; use the reporting checklist on a real workflow; compare original and rerun configurations; connect discrepancies to the final-project reproducibility record.

[See the provisional Week 12 Python plan.](#)

30.5 Development status

This is a planning page. The finished session will be developed with Week 11 so the two research-agent meetings form one coherent arc.

31 Week 12 Python — Rerunning and auditing an agent trajectory

[Return to the Week 12 readings outline.](#)

This page records the coding direction only. The full walkthrough, notebook and completion task will be added after the week's literature, slides and examples have been developed together.

31.1 Planned code trace

Research task: Rerun a three-stage research routine and compare its saved events with an earlier record.

Python sequence: JSON files, bounded loops, stopping after three calls and field-by-field discrepancy records. The runnable notebook follows input → messages → route → call → raw return → parsed output → check.

31.2 Planned completion recording

Show the saved three-event rerun, make the temperature change and explain which fields were fixed, which changed and which causal explanation remains unsupported. Submit one narrated screen recording using the [recording and submission instructions](#) by **5:00 p.m. Eastern on the Tuesday before the next class**. The key must not appear.

31.3 Development status

The page is deliberately provisional. It does not yet publish an assessed notebook or task.

32 Week 13 — What culture, ideology and hierarchy are encoded in models?

32.1 Question

How can we identify patterned differences in model output without immediately assigning them a social cause?

32.2 Why this week belongs here

After treating models as research infrastructure, the course turns them into objects of sociological inquiry. The same measurement and causal-design standards still apply.

32.3 Provisional core readings

- Barrie et al. (2026), §10.6, especially §§10.6.1–10.6.4. [Download chapter PDF](#). (Barrie et al. 2026)
- Waight et al. (2026), “State Media Control Influences Large Language Models.” (Waight et al. 2026)
- Buyl et al. (2026), “Large Language Models Reflect the Ideology of Their Creators.” (Buyl et al. 2026)
- Further cases: Kim et al. (2026) and Boelaert et al. (2025). (Kim, Street, et al. 2026; Boelaert et al. 2025)

32.4 Planned chapter and class arc

Connect model audits to established audit studies; reconstruct the central papers; distinguish output patterns from explanations about training, culture or alignment; design a factorial audit with predeclared alternatives.

[See the provisional Week 13 Python plan.](#)

32.5 Development status

This is a planning page. Readings and examples remain provisional until the literature-led build and slide review are complete.

33 Week 13 Python — A small model audit

[Return to the Week 13 readings outline.](#)

This page records the coding direction only. The full walkthrough, notebook and completion task will be added after the week's literature, slides and examples have been developed together.

33.1 Planned code trace

Research task: Compare two prompt frames across hosted and local models while preserving every condition and avoiding unsupported cultural explanations.

Python sequence: nested loops, parameter grids, lists of records and a table created only after the plain records are understood. The runnable notebook follows input → messages → route → call → raw return → parsed output → check.

33.2 Planned completion recording

Trace one cell through both loops, make the role-word change, present one bounded observed difference and name at least two unresolved explanations. Submit one narrated screen recording using the [recording and submission instructions](#) by **5:00 p.m. Eastern on the Tuesday before the next class**. The key must not appear.

33.3 Development status

The page is deliberately provisional. It does not yet publish an assessed notebook or task.

34 Week 14 — Project presentations

This meeting is reserved for student project presentations. There is no new lecture content, required reading, coding walkthrough or weekly completion task.

34.1 Presentation format

Each student has 10–12 minutes to present, followed by questions and brief feedback from the group. Presentations should state the sociological question, proposed evidence and research design, where an LLM enters the project, what would need validating, and the main limitation or unresolved decision.

34.2 What to bring

Bring the presentation file and a backup copy. The final research-design package is submitted separately during the examination period, after you have considered the feedback received in class.

35 References

This bibliography consolidates works cited across the book. Weekly chapters link directly to the assigned DOI, publisher or stable author page.

- Alvero, AJ, Dustin S. Stoltz, Oscar Stuhler, and Marshall A. Taylor. 2026. “Generative AI in Sociological Research: State of the Discipline.” *Sociological Science* 13: 45–62. <https://doi.org/10.15195/v13.a3>.
- Ashery, Ariel Flint, Luca Maria Aiello, and Andrea Baronchelli. 2025. “Emergent Social Conventions and Collective Bias in LLM Populations.” *Science Advances* 11 (20): eadu9368. <https://doi.org/10.1126/sciadv.adu9368>.
- Ashokkumar, Ashwini, Luke Hewitt, Isaias Ghezae, and Robb Willer. 2026. “Large Language Models Can Predict the Results of Social Science Experiments.” *Nature*. <https://doi.org/10.1038/s41586-026-10742-x>.
- Bai, Hui, Jan G. Voelkel, Shane Muldowney, Johannes C. Eichstaedt, and Robb Willer. 2025. “LLM-Generated Messages Can Persuade Humans on Policy Issues.” *Nature Communications* 16: 6037. <https://doi.org/10.1038/s41467-025-61345-5>.
- Barrie, Christopher, Lisa P. Argyle, James Bisbee, Michael Heseltine, Christopher Lucas, Jon Mellon, Alexis Palmer, Margaret Roberts, and Arthur Spirling. 2026. “AI and Research Methods.” In *Artificial Intelligence, Politics, and Political Science*, edited by Nathaniel Persily and Joshua A. Tucker, 313–64. Cambridge University Press. <https://doi.org/10.33774/apsa-2026-h59kk>.
- Barrie, Christopher, Elli Palaiologou, and Petter Törnberg. 2026. “Prompt Stability Scoring for Text Annotation with Large Language Models.” <https://doi.org/10.48550/arXiv.2407.02039>.
- Barrie, Christopher, Alexis Palmer, and Arthur Spirling. 2025. “Replication for Language Models: Problems, Principles, and Best Practices for Political Science.” https://arthurspirling.org/documents/BarriePalmerSpirling_TrustMeBro.pdf.
- Barrie, Christopher, and Petter Törnberg. 2025. “Emergent LLM Behaviors Are Observationally Equivalent to Data Leakage.” <https://arxiv.org/abs/2505.23796>.
- Bisk, Yonatan, Ari Holtzman, Jesse Thomason, Jacob Andreas, Yoshua Bengio, Joyce Chai, Mirella Lapata, et al. 2020. “Experience Grounds Language.” In *Proceedings of EMNLP*, 8718–35. <https://aclanthology.org/2020.emnlp-main.703/>.
- Bloemraad, Irene, Fabiana Silva, and Kim Voss. 2016. “Rights, Economics, or Family? Frame Resonance, Political Ideology, and the Immigrant Rights Movement.” *Social Forces* 94 (4): 1647–74. <https://doi.org/10.1093/sf/sov123>.
- Boelaert, Julien, Samuel Coavoux, Étienne Ollion, Ivaylo Petev, and Patrick Präg. 2025. “Machine Bias: How Do Generative Language Models Answer Opinion Polls?” *Sociological Methods & Research* 54 (3): 1156–96. <https://doi.org/10.1177/00491241251330582>.
- Buyl, Maarten, Alexander Rogiers, Sander Noels, Guillaume Bied, Iris Dominguez-Catena, Edith Heiter, Iman Johary, et al. 2026. “Large Language Models Reflect the Ideology of Their Creators.” *Npj Artificial Intelligence* 2 (1): 7. <https://doi.org/10.1038/s44387-025-00048-0>.
- Chae, Youngjin, and Thomas Davidson. 2026. “Large Language Models for Text Classification:

- From Zero-Shot Learning to Instruction-Tuning.” *Sociological Methods & Research* 55 (2): 501–67. <https://doi.org/10.1177/00491241251325243>.
- Chuang, Yun-Shiuan, Agam Goyal, Nikunj Harlalka, Siddharth Suresh, Robert Hawkins, Sijia Yang, Dhavan Shah, Junjie Hu, and Timothy Rogers. 2024. “Simulating Opinion Dynamics with Networks of LLM-Based Agents.” In *Findings of the Association for Computational Linguistics: NAACL 2024*, 3326–46. <https://doi.org/10.18653/v1/2024.findings-naacl.211>.
- Collins, Randall. 2013. “Entering and Leaving the Tunnel of Violence: Micro-Sociological Dynamics of Emotional Entrainment in Violent Interactions.” *Current Sociology* 61 (2): 132–51. <https://doi.org/10.1177/0011392112456500>.
- Dafoe, Allan, Baobao Zhang, and Devin Caughey. 2018. “Information Equivalence in Survey Experiments.” *Political Analysis* 26 (4): 399–416. <https://doi.org/10.1017/pan.2018.9>.
- Davidson, Thomas, and Daniel Karell. 2025. “Integrating Generative Artificial Intelligence into Social Science Research: Measurement, Prompting, and Simulation.” *Sociological Methods & Research* 54 (3): 775–93. <https://doi.org/10.1177/00491241251339184>.
- Feuerriegel, Stefan, Christopher Barrie, M. J. Crockett, Laura K. Globig, Killian L. McLoughlin, Dan-Mircea Mirea, Arthur Spirling, Diyi Yang, et al. 2026. “A Reporting Checklist for Large Language Models in Behavioural Science.” *Nature Human Behaviour* 10: 1182–86. <https://doi.org/10.1038/s41562-026-02492-7>.
- Geiecke, Friedrich, and Xavier Jaravel. 2026. “The Rise of AI-Led Interviews in Social Science.” https://benjamin.geiecke.com/files/ai_interviews.pdf.
- Goodwin, Charles. 1994. “Professional Vision.” *American Anthropologist* 96 (3): 606–33. <https://doi.org/10.1525/aa.1994.96.3.02a00100>.
- Ibrahim, Elida Izani, and Andrea Voyer. 2026. “Qualitative Research with LLM Chatbots: Technological Reflexivity for Interpretative Technology.” *Qualitative Research*. <https://doi.org/10.1177/14687941251390794>.
- Igarashi, Akira, Yoshinobu Kano, and Hirofumi Miwa. 2025. “ChatGPT Versus Humans in Judging Discriminatory Scenarios: Experimental Evidence from a Japanese Context.” *Humanities and Social Sciences Communications* 12: 1776. <https://doi.org/10.1057/s41599-025-06054-6>.
- Jack, Tullia, Alex Cooper, and Lisa Flower. 2026. “Automating the Qualitative Interview? Using Gen AI Chatbots in Social Science Research.” *Methodological Innovations*. <https://doi.org/10.1177/20597991261448157>.
- Kim, Junsol, Shiyang Lai, Nino Scherrer, Blaise Agüera y Arcas, and James Evans. 2026. “Reasoning Models Generate Societies of Thought.” <https://arxiv.org/abs/2601.10825>.
- Kim, Junsol, and Byungkyu Lee. 2026. “AI-Augmented Surveys: Leveraging Large Language Models and Surveys for Opinion Prediction.” <https://doi.org/10.48550/arXiv.2305.09620>.
- Kim, Junsol, Winnie Street, Roberta Rocca, Diane M. Korngiebel, Adam Waytz, James Evans, and Geoff Keeling. 2026. “Inducing Language Models to Assert Their Own Consciousness Restores Human Beliefs and Values.” <https://arxiv.org/abs/2607.28607>.
- Kozlowski, Austin C., and James A. Evans. 2025. “Simulating Subjects: The Promise and Peril of Artificial Intelligence Stand-Ins for Social Agents and Interactions.” *Sociological Methods & Research* 54 (3): 1017–73. <https://doi.org/10.1177/00491241251337316>.
- Lax, Jeffrey R., and Justin H. Phillips. 2009. “Gay Rights in the States: Public Opinion and Policy Responsiveness.” *American Political Science Review* 103 (3): 367–86. <https://doi.org/10.1017/S0003055409990050>.
- Lu, Chris, Cong Lu, Robert Tjarko Lange, Yutaro Yamada, Shengran Hu, Jakob Foerster, David Ha, and Jeff Clune. 2026. “Towards End-to-End Automation of AI Research.” *Nature* 651: 914–19. <https://doi.org/10.1038/s41586-026-10265-5>.
- Macy, Michael W., and Robert Willer. 2002. “From Factors to Actors: Computational Sociology

- and Agent-Based Modeling.” *Annual Review of Sociology* 28: 143–66. <https://doi.org/10.1146/annurev.soc.28.110601.141117>.
- Merton, Robert K., and Patricia L. Kendall. 1946. “The Focused Interview.” *American Journal of Sociology* 51 (6): 541–57. <https://doi.org/10.1086/219886>.
- Nassauer, Anne, and Nicolas M. Legewie. 2021. “Video Data Analysis: A Methodological Frame for a Novel Research Trend.” *Sociological Methods & Research* 50 (1): 135–74. <https://doi.org/10.1177/0049124118769093>.
- Nature Machine Intelligence. 2026. “Multi-Agent AI Systems Need Transparency.” *Nature Machine Intelligence* 8: 1. <https://doi.org/10.1038/s42256-026-01183-2>.
- Park, Joon Sung, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. “Generative Agents: Interactive Simulacra of Human Behavior.” In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 1–22. <https://doi.org/10.1145/3586183.3606763>.
- Rivera, Lauren A. 2012. “Hiring as Cultural Matching: The Case of Elite Professional Service Firms.” *American Sociological Review* 77 (6): 999–1022. <https://doi.org/10.1177/0003122412463213>.
- Stuhler, Oscar, Cat Dang Ton, and Étienne Ollion. 2025. “From Codebooks to Promptbooks: Extracting Information from Text with Generative Large Language Models.” *Sociological Methods & Research* 54 (3): 794–848. <https://doi.org/10.1177/00491241251336794>.
- Than, Nga, Leanne Fan, Tina Law, Laura K. Nelson, and Leslie McCall. 2025. “Updating ‘the Future of Coding’: Qualitative Coding with Generative Large Language Models.” *Sociological Methods & Research* 54 (3): 849–88. <https://doi.org/10.1177/00491241251339188>.
- Timmermans, Stefan, and Iddo Tavory. 2012. “Theory Construction in Qualitative Research: From Grounded Theory to Abductive Analysis.” *Sociological Theory* 30 (3): 167–86. <https://doi.org/10.1177/0735275112457914>.
- Velez, Yamil Ricardo. 2025. “Crowdsourced Adaptive Surveys.” *Political Analysis* 33 (4): 284–97. <https://doi.org/10.1017/pan.2024.34>.
- Viterna, Jocelyn S. 2006. “Pulled, Pushed, and Persuaded: Explaining Women’s Mobilization into the Salvadoran Guerrilla Army.” *American Journal of Sociology* 112 (1): 1–45. <https://doi.org/10.1086/502690>.
- Waight, Hannah, Eddie Yang, Yin Yuan, Solomon Messing, Margaret E. Roberts, Brandon M. Stewart, et al. 2026. “State Media Control Influences Large Language Models.” *Nature* 655: 685–93. <https://doi.org/10.1038/s41586-026-10506-7>.
- Wallerstein, Lisa. 2009. “25 Years of Factorial Surveys in Sociology: A Review.” *Social Science Research* 38 (3): 505–20. <https://doi.org/10.1016/j.ssresearch.2009.03.004>.
- Wang, Angelina, Jamie Morgenstern, and John P. Dickerson. 2025. “Large Language Models That Replace Human Participants Can Harmfully Misportray and Flatten Identity Groups.” *Nature Machine Intelligence*. <https://doi.org/10.1038/s42256-025-00986-z>.
- Xie, Yueqi, Lemeng Liang, Shuzhen Li, Yifu Lu, Zhiwen Xiao, Mengdi Shi, Junming Huang, Mengdi Wang, and Yu Xie. 2026. “Evaluating the Statistical Realism of LLM-Generated Social Science Data.” *Proceedings of the National Academy of Sciences* 123 (19): e2538145123. <https://doi.org/10.1073/pnas.2538145123>.